



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064



REAL-TIME PUBLIC TRANSPORT TRACKING SYSTEM FOR SMALL CITIES

A PROJECT REPORT

Submitted by

KEERTHI.G- 20221CSG0078

KUMUDH RAO HARALE M- 20221CSG0052

Under the guidance of,

Ms.SHET RESHMA PRAKASH

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND TECHNOLOGY

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064



PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this project titled “**REAL-TIME PUBLIC TRANSPORT TRACKING FOR SMALL CITIES**” is a Bonafide work of “KEERTHI.G (20221CSG0078), KUMUDH RAO HARALE M(20221CSG0052)”, who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND TECHNOLOGY during 2025-26.

Ms. Shet Reshma Prakash
Project Guide
Assistant Professor
PSCS
Presidency University

Mrs. Benitha Christinal J
Program Project
Coordinator
PSCS
Presidency University

Dr. Sampath A K
School Project
Coordinator
PSCS
Presidency University

Dr. Pallavi R
Head of the Department
PSCS
Presidency University

Dr. Duraipandian N
Dean
PSCS
Presidency University

Examiners

Sl. No	Name	Signature	Date
1			
2			

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We the students of final year B.Tech in COMPUTER SCIENCE AND TECHNOLOGY at Presidency University, Bengaluru, named “KEERTHI.G (20221CSG0078), KUMUDH RAO HARALE M(20221CSG0052)”, hereby declare that the project titled “**REAL-TIME PUBLIC TRANSPORT TRACKING FOR SMALL CITIES**” has been independently carried out by us and submitted in partial fulfilment for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution

KEERTHI.G

USN: 20221CSG0078

KUMUDH RAO HARALE M USN: 20221CSG0052

PLACE: BENGALURU

DATE: 24- April-2026

ACKNOWLEDGEMENT

For completing this project work, We/I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. We extend our gratitude to our beloved **Chancellor, Vice Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the project.

I would like to sincerely thank my internal guide Ms. **Shet Reshma Prakash, Assistant Professor**, Presidency School of Computer Science and Engineering, Presidency University, for his/her moral support, motivation, timely guidance and encouragement provided to us during the period of our project work.

I am also thankful to **Dr. Pallavi, Professor, Head of the Department, Presidency School of Computer Science and Engineering**, Presidency University, for his mentorship and encouragement.

We express our cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS, Presidency School of Computer Science and Engineering and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my project work.

We are grateful to **Dr. Sampath A K, and Ms. Benitha Christinal J**, PSCS Project Coordinators, for facilitating problem statements, coordinating reviews, monitoring progress and providing their valuable support and guidance.

We are also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

KEERTHI.G

KUMUDH RAO HARALE M

Abstract

Public transportation systems which operate in cities need to operate their services for bus passengers because people need real-time information about bus locations and status to travel. The system problems make the system less reliable which results in a negative impact on user satisfaction. The existing problem needs a solution through a Real-Time Public Transport Tracking System which will enhance both operational visibility and system performance. The system development uses web technologies to create a tracking system which functions as if it uses actual GPS equipment. The interactive map interface uses Leaflet and OpenStreetMap to display bus movements. The backend uses Node.js and Express while Socket.IO enables real-time communication between the server and users. The system models multiple buses which operate with different parameters that include location speed route capacity and status which get updated through instant sharing. The system provides an admin panel which handles fleet operations together with a driver dashboard that manages trip activities. The system shows its ability to track actual time by using simulated data which allows users to receive updates while their information stays synchronized. The system provides better visibility and user experience while creating a scalable system which can connect with future GPS tracking systems.

Table of Content

Sl. No.	Title	Page No.
	Declaration	
	Acknowledgement	iii
	Abstract	iii
	List of Figures	iii
	List of Tables	iii
	Abbreviations	iii
1.	Introduction 1.1 Background 1.2 Statistics of project 1.3 Prior existing technologies 1.4 Proposed approach 1.5 Objectives 1.6 SDGs 1.7 Overview of project report	1
2.	Literature review	6
3.	Methodology	13
4.	Project management 4.1 Project timeline 4.2 Risk analysis	20
5.	Analysis and Design 5.1 Requirements 5.2 Block Diagram 5.3 System Flow Chart 5.4 Choosing devices 5.5 Designing units 5.6 Standards 5.7 Mapping with IoTWF reference model layers 5.8 Domain model specification	25

	5.9 Communication model 5.10 IoT deployment level 5.11 Functional view 5.12 Mapping IoT deployment level with functional view 5.13 Operational view 5.14 Other Design	
6.	Hardware, Software and Simulation 6.1 Hardware 6.2 Software development tools 6.3 Software code 6.4 Simulation	42
7.	Evaluation and Results 7.1 Test points 7.2 Test plan 7.3 Test result 7.4 Insights	48
8.	Social, Legal, Ethical, Sustainability and Safety Aspects 8.1 Social aspects 8.2 Legal aspects 8.3 Ethical aspects 8.4 Sustainability aspects 8.5 Safety aspects	56
9.	Conclusion	61
10..	References	63
11.	Appendix	64

List of Figures

Figure No.	Name of the Figure	Page No.
Fig 1.1	Sustainable Development Goals	4
Fig 3.1	System Architecture	14
Fig 3.2	Use Case Diagram	16
Fig 3.3	System Data Flow	17
Fig 3.4	SDLC Phases	19
Fig 4.1	Project Timeline Chart	20
Fig 5.1	Block Diagram	29
Fig 5.2	Flow Chart	32
Fig 7.1	System Test Points Architecture	49
Fig 7.2	Real-Time Update Latency	53
Fig 7.3	System Performance Under Multiple Updates	53

List of Tables

Table No.	Name of the Table	Page No.
Table 2.1	Summary of Literature Review	10
Table 4.2	PESTEL Analysis	22
Table 4.3	Project Phase Risk Matrix	24
Table 7.1	System Observation Table	49

Abbreviations

Abbreviation	Full Form
GPS	Global Positioning System
IoT	Internet of Things
API	Application Programming Interface
UI	User Interface
UX	User Experience
SDLC	Software Development Life Cycle
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
HTTP	Hyper Text Transfer Protocol
TCP	Transmission Control Protocol
CORS	Cross-Origin Resource Sharing
DB	Database
ETA	Estimated Time of Arrival
SDG	Sustainable Development Goals

Chapter 1

Introduction

Transportation is an essential part of urban life, but traditional public transport systems often face challenges such as unpredictable schedules, lack of real-time information, and inefficient management. These issues create inconvenience for passengers and reduce the overall effectiveness of transport services. With the growth of cities and increasing demand for mobility, there is a need for smarter and more reliable transport solutions that can provide accurate and timely information. This project presents a real-time bus tracking and management system that uses modern web technologies to improve transport efficiency. By leveraging GPS-based simulation and real-time communication using Node.js and Socket.IO, the system provides live updates on bus location, speed, and status. It also includes features for fleet monitoring, administrative control, and driver interaction, making the system more dynamic and responsive. The solution aims to enhance user experience, reduce uncertainty, and support the development of smarter and more sustainable transportation systems.

1.1 Background

Public transportation serves as a vital component for enabling urban mobility during specific periods in developing cities, where a major part of the city population relies on bus services for their daily commuting needs. The traditional public transportation systems operating in the city fail to provide passengers with real-time tracking systems, which would enable them to determine the current location of buses, their expected arrival times, and their operational status. This situation leads to passengers experiencing longer waiting periods, which brings about unpredictable situations, and results in lower transportation efficiency. The development of real-time tracking solutions through web technologies and smart systems enables transportation services to achieve better operational results. Many modern systems use GPS-based tracking; however, implementing such systems requires additional hardware, increasing cost and complexity. This project focuses on developing a cost-effective alternative by simulating real-time tracking using web technologies, making it suitable for small cities and educational purposes.

1.2 Statistics

The urban population growth requires cities to establish effective public transportation systems. According to global transportation studies, more than 55% of the world's population lives in urban areas, and this number is expected to rise further. The public bus system in Bengaluru operates services for millions of passengers each day, yet the system encounters challenges because of delays and the absence of up-to-date information. Studies indicate that real-time tracking systems can reduce passenger waiting time by up to 20 to 30 percent while enhancing overall system performance. The small cities in the area fail to implement those systems because they need expensive infrastructure development. The project demonstrates the need for practical solutions that can be implemented at low cost and expanded to serve larger operations.

1.3 Prior existing technologies

The current systems for public transport tracking depend on GPS hardware equipment which vehicle operators use for tracking purposes. The systems send their location information to central servers which create displayable content for users through their mobile and web platforms. The implementation process typically uses GSM technology together with IoT sensors and cloud computing platforms. The systems deliver precise real-time information but require expensive initial setup and ongoing operational costs. The system's operation depends on physical equipment which creates problems through both equipment failure and loss of signals. Google Maps and similar applications offer transit tracking services but they need outside data for their operation and lack options to adapt to specific transportation networks.

1.4 Proposed approach

1.4.1 Aim of the Project

To design and develop a web-based real-time public transport tracking system that simulates live bus tracking using efficient and scalable technologies.

1.4.2 Motivation

The motivation behind this project is to provide a low-cost and flexible solution for transport monitoring, especially suitable for small cities or prototype development without requiring physical GPS hardware.

1.4.3 Proposed System

The system uses:

- Leaflet + OpenStreetMap for map visualization
- Node.js + Express for backend
- Socket.IO for real-time communication

Instead of real GPS, bus movement is simulated dynamically, and updates are broadcast to users in real time.

1.4.4 Applications

- Public transport monitoring
- Smart city systems
- Fleet management
- Educational and prototype systems

1.4.5 Limitations

- Uses simulated data instead of real GPS
- Accuracy depends on simulation logic
- Not directly deployable without hardware integration

1.5 Objectives

- ☐ To design a system capable of simulating real-time bus location updates dynamically.
- ☐ To implement real-time communication between server and users using Socket.IO.
- ☐ To develop an interactive map interface for visualizing transport data.
- ☐ To create an admin module for managing buses and system operations.
- ☐ To ensure scalability and easy integration with real GPS systems in future.

1.6 SDGs

This project aligns with the following United Nations Sustainable Development Goals:

SDG 9 (Industry, Innovation and Infrastructure):

This project promotes innovation by integrating smart technologies into transport systems. It helps modernize infrastructure through data-driven solutions and real-time monitoring. Such advancements improve efficiency, reliability, and scalability in transportation networks. Overall, it supports the development of resilient and future-ready infrastructure.

SDG 11 (Sustainable Cities and Communities):

The project enhances urban mobility by making transport systems more efficient and accessible. It helps reduce traffic congestion and improves the daily commuting experience. By encouraging better use of public transport, it contributes to more organized and livable cities. This leads to safer and more sustainable urban environments.

SDG 13 (Climate Action):

This project helps reduce fuel consumption by optimizing routes and minimizing unnecessary travel. Lower fuel use directly leads to a reduction in harmful emissions. It also encourages more sustainable transport habits among users. In the long run, it supports efforts to combat climate change and protect the environment.



Fig 1.1 Sustainable development goals

1.7 Overview of project report

This report is organized into multiple chapters describing the development and implementation of the proposed system. Chapter 1 introduces the background, objectives, and approach of the project. Chapter 2 presents the literature review of existing systems and technologies. Chapter 3 explains the methodology adopted for system development. Chapter 4 discusses project planning, timeline, and risk analysis. Chapter 5 covers system analysis and design. Chapter 6 describes hardware, software, and simulation details. Chapter 7 presents evaluation and results. Chapter 8 discusses social, legal, ethical, and sustainability aspects. Finally, Chapter 9 concludes the report and suggests future improvements.

Chapter 2

Literature review

Research on intelligent transportation systems, particularly real-time bus tracking and fleet management, has grown significantly with the advancement of GPS, IoT, and web-based communication technologies. Early systems mainly focused on basic GPS tracking and location display, while recent developments emphasize real-time communication, system integration, and user interaction. Technologies such as WebSocket's, cloud computing, and mobile applications have enabled continuous data exchange, making transport systems more responsive and efficient. There is also a clear shift towards integrating tracking with management features like fleet control, passenger information systems, and driver interaction modules to improve overall system performance.

Modern studies highlight the importance of real-time updates, scalability, and system reliability in public transport solutions. Existing systems mainly rely on GPS-based tracking (Singh and Kaur, 2021), whereas this project uses simulated real-time data for cost-effective development. Many approaches combine GPS tracking with centralized monitoring systems to provide live updates on vehicle location, speed, and status. However, challenges such as network dependency, data latency, scalability, and lack of predictive capabilities are still common. Recent trends suggest moving towards event-driven architectures, automated systems, and integration of advanced analytics to enhance decision-making and user experience.

2.1 Key Studies & Findings

2.1.1 GPS-Based Bus Tracking Systems

Several studies, including works by Babu et al. (2021) and Gayathri et al. (2018), focus on GPS-based bus tracking systems that provide real-time location updates to passengers. These systems improve user convenience by reducing uncertainty in arrival times and enabling better travel planning. Most implementations use a centralized server to collect and distribute location data through mobile or web interfaces. While effective, these systems often face limitations related to GPS accuracy and network reliability, especially in densely populated urban areas.

2.1.2 Use of Smartphones and Low-Cost Tracking Approaches

Research by Mane and Khairnar (2014) highlights the use of smartphone-based GPS tracking as a cost-effective alternative to dedicated hardware systems. This approach simplifies deployment and reduces system cost while maintaining tracking functionality. However, such systems depend heavily on mobile device performance, battery life, and internet connectivity. Despite these limitations, they demonstrate the feasibility of lightweight and scalable tracking solutions, which influence modern web-based implementations.

2.1.3 Real-Time Communication and Web-Based Systems

Recent studies emphasize the role of real-time communication technologies in transport systems. Approaches using web-based platforms and event-driven architectures enable instant updates between the server and users. Systems described by Ogbonna et al. (2016) and Gogri et al. (2020) show how continuous data transmission improves fleet monitoring and passenger information services. However, maintaining real-time communication requires efficient handling of data flow and server resources. This highlights the need for technologies like WebSocket's, which support continuous bidirectional communication.

2.1.4 Smart Bus Management and Administrative Control

Advanced systems, such as those proposed by Madhur et al. (2018), integrate tracking with management features like fleet monitoring, status updates, and administrative controls. These systems allow operators to manage buses more effectively by adding, removing, or updating vehicle status in real time. While this improves operational efficiency, many systems rely on manual input from drivers or administrators, which can lead to inaccuracies. Automation and real-time synchronization are suggested as improvements.

2.2 Integration of Data and System Scalability

Many modern transportation systems focus on handling large volumes of real-time data generated by multiple vehicles. Studies show that centralized and cloud-based systems can manage such data efficiently, enabling scalability across larger networks. However, challenges such as latency, data consistency, and system performance remain significant. These limitations

highlight the need for optimized architectures that can handle frequent updates without affecting system responsiveness.

2.3 Role of Real-Time Event-Driven Technologies

Event-driven communication models are increasingly used in modern transport systems to provide instant updates. Technologies such as WebSocket's allow continuous communication between server and clients, ensuring that users receive live updates without repeated requests. This approach reduces delay and improves system responsiveness. Compared to traditional request-response models, event-driven systems are more suitable for applications requiring constant data updates, such as real-time bus tracking.

2.4 Gaps Identified in Existing Systems

Although significant progress has been made, several gaps still exist in current systems. Many solutions lack predictive capabilities, such as estimating delays based on traffic conditions. Security and data privacy are also often overlooked. Additionally, scalability remains a challenge when systems are deployed in large urban environments. Another limitation is the dependency on manual inputs, which can affect data accuracy. Addressing these gaps requires integrating automation, advanced analytics, and more robust system architectures.

2.5 Relevance to the Proposed Project

The proposed real-time bus tracking system builds upon these existing studies by combining GPS-based simulation with real-time communication using Socket.IO. Unlike traditional systems, it emphasizes continuous data updates, interactive dashboards, and role-based controls for administrators and drivers. The project also addresses some of the identified gaps by implementing an event-driven architecture that ensures faster and more efficient data transmission. This approach aligns with current trends in intelligent transportation systems and contributes to improving urban mobility and transport management.

Summary of Literatures reviewed

Table 2.1 Summary of Literature reviews

S L	Article Title, published year, Journal name	Methods	Key features	Merits	Demerits	Relevance to the project
1	J. Babu et al., “ <i>GPS Based Bus Tracking System</i> ,” IJERT, 2021	GPS tracking; central server	Real-time bus location updates	Real-time visibility; punctuality; easy access; simple system	Depends on GPS and network	Forms base for real-time tracking in project
2	Y. H. A. Abdelhafed, “ <i>Real Time Bus Tracking using GPS</i> ,” IJRASET, 2025	GPS tracking; centralized system	Improves transport coordination	Better scheduling; monitoring; efficiency; live updates	Scalability issues	Similar to centralized admin control in project
3	G. Gayathri et al., “ <i>Real Time Bus Tracking System</i> ,” IJARCS, 2018	GPS + mobile app	Continuous tracking and communication	User-friendly; real-time access; better planning; communication	High data load	Related to Socket.IO real-time updates
4	P. S. Mane & V. D. Khairnar, “ <i>Bus Tracking Using GPS on Smartphones</i> ”	Smartphones GPS	Low-cost tracking system	Cost-effective; flexible; easy deployment ; no extra hardware	Battery dependency	Supports lightweight implementation idea

	,” IOSR, 2014					
5	R. Khopde et al., “ <i>Public Transport Tracking System</i> ,” IERJ, 2017	GPS tracking ;route system	Provides route and timing info	Reduces uncertainty; convenience; planning; simple design	No traffic integration	Aligns with route-based tracking
6	J. C. Ogbonna et al., “ <i>Real-Time Vehicle Tracking System</i> ,” IJCTT, 2016	GPS + wireless comm.	Improves fleet monitoring	Accurate tracking; transparency; control; reliability	Data latency	Supports real-time communication logic
7	I. Madhur et al., “ <i>Smart Bus Tracking & Management System</i> ,” IRE, 2018	GPS + admin control	Combines tracking and management	Centralized control; coordination; flexibility; monitoring	Manual input errors	Matches admin & driver modules
8	S. Eken & A. Sayar, “ <i>Smart Bus Tracking using QR Codes</i> ,” IEEE INISTA, 2014	QR + location services	Provides bus info via QR	Innovative; easy access; low cost; user-friendly	Needs user interaction	Useful for future enhancements

9	B. Thomas et al., <i>"Intelligent Location Tracker,"</i> IJERT, 2021	GPS tracking	Improves route monitoring	Efficient routing; better tracking; performance; reliability	No prediction	Indicates need for predictive features
10	K. Gogri et al., <i>"Real Time Bus Tracking System,"</i> IJERT, 2020	GPS + real-time updates	Enhances passenger info	Continuous updates; reliable; simple; user satisfaction	Scalability issues	Supports fleet update feature
11	H. Khaled et al., <i>"Intelligent Transport Tracking,"</i> Qeios, 2023	Multi-tech integration	Efficient tracking system	Advanced; scalable; flexible; efficient monitoring	Limited implementation	Encourages advanced integration
12	A. Bessa et al., <i>"RioBusData,"</i> arXiv, 2016	Data analysis	Detects route anomalies	Data insights; efficiency; analytics; accuracy	Complex processing	Useful for analytics extension
13	K. Nguyen et al., <i>"GPS Bus Data Analysis,"</i> arXiv, 2019	GPS trajectory analysis	Optimizes routes	Better planning; optimization; insights; efficiency	Not real - time	Suggests future improvements

1 4	K. Ganesh et al., “ <i>Passenger Information System</i> ,” arXiv, 2012	Real-time system	Provides arrival updates	Improves experience; accessible; reliable; reduces wait	Infrastructure dependency	Supports frontend dashboard
1 5	M. Z. Parvez et al., “ <i>GSM-Based Tracking System</i> ,” IEEE, 2010	GSM communication	Cost - effective tracking	Low cost; wide coverage; simple; easy implementation	Data delay	Alternative communication method
1 6	T. Le-Tien & V. P. The, “ <i>Routing & Tracking System</i> ,” IEEE, 2010	Routing algorithms	Improves navigation	Better routing; structured; control; efficiency	Static routing	Shows need for dynamic routing
1 7	W. El-Medany et al., “ <i>GPS/GPRS Tracking System</i> ,” IEEE, 2010	GPS+ GPRS	Affordable tracking	Cost-effective; scalable; efficient; practical	Security issues	Highlights need for data security
1 8	G. Mintsis et al., “ <i>GPS in Transportation</i> ,” European Journal, 2004	GPS analysis	Improves efficiency & safety	Foundational; widely applicable; safety; efficiency	Outdated	Provides basic concept foundation

Chapter 3

Methodology

3.1 Introduction

This chapter explains the overall approach used to design and develop the real-time bus tracking and management system. The methodology focuses on building a system that can simulate and monitor multiple buses while providing continuous updates to users. The design follows a modular and event-driven approach, ensuring that each component of the system performs a specific function while remaining connected through real-time communication.

The system is developed using modern web technologies, with a clear separation between backend processing and frontend interaction. The backend is responsible for managing data, handling requests, and broadcasting updates, while the frontend provides a user interface for visualization and control. The use of real-time communication ensures that any change in the system is immediately reflected across all connected users.

3.2 System Architecture

The proposed system follows a client-server architecture. It consists of three main components: the server (backend), the clients (frontend dashboards), and the communication layer.

- **Backend Server:** Handles data processing, maintains the bus dataset, and manages communication between clients.
- **Frontend Clients:** Include the user interface for administrators and drivers to interact with the system.
- **Communication Layer:** Uses Socket.IO to enable real-time data exchange between server and clients.

The architecture is designed to support multiple users simultaneously. Whenever there is a change in bus data, such as location or status, the server broadcasts the update to all connected clients. This ensures synchronization across the system.

Figure 3.1 illustrates how different users such as administrators, drivers, and passengers interact with the frontend interface to access system features. The frontend acts as a bridge, sending user actions and requests to the backend server. Through Socket.IO, a real-time communication channel is established, allowing the server to instantly push updates like bus location and status. As a result, users receive continuous, live information without needing to refresh the system manually.

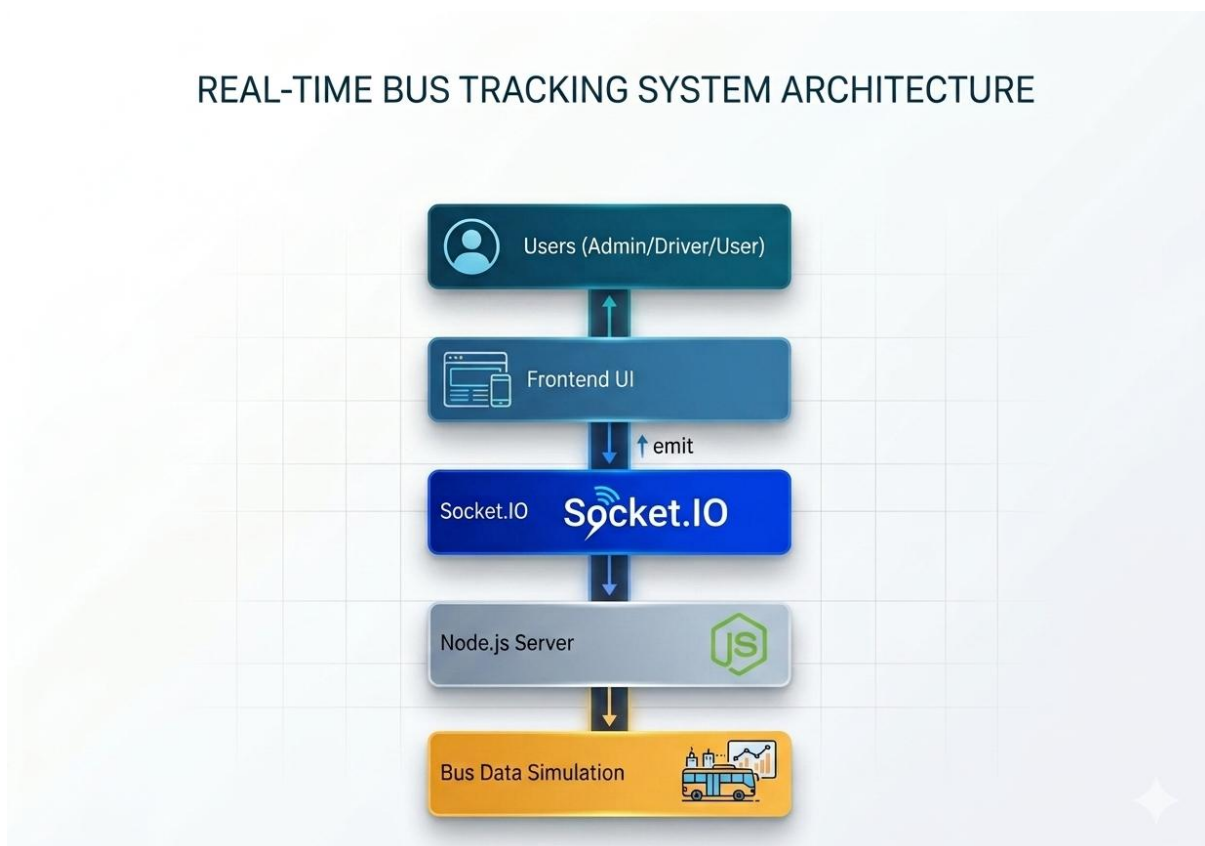


Fig 3.1 System Architecture

3.3 Technologies Used

The implementation of the system is based on the following technologies:

- Node.js: Used for building the backend server due to its non-blocking and event-driven nature.

- Express.js: Provides a lightweight framework for handling server requests and routing.
- Socket.IO: Enables real-time bidirectional communication between server and clients.
- HTML, CSS, JavaScript: Used to design the frontend interface and dashboards.
- CORS: Allows communication between frontend and backend running on different origins.

These technologies are chosen because they support scalability, real-time interaction, and efficient handling of multiple connections.

3.4 Working of the System

The system operates by simulating bus movement and continuously updating relevant parameters such as location, speed, and capacity. Initially, a set of buses is defined with predefined attributes including ID, latitude, longitude, route, and status.

A timer-based function updates the position of each bus at regular intervals. The latitude and longitude values are slightly modified to simulate movement, while speed and capacity are randomly adjusted to represent real-world variations. After updating the data, the server sends this information to all connected clients.

The frontend receives these updates and displays them in real time. This allows users to track bus movement dynamically. The system ensures that all users see consistent and up-to-date information.

3.5 Real-Time Communication Mechanism

A key aspect of this project is the use of real-time communication. Instead of relying on repeated requests from the client, the system uses an event-driven model where the server pushes updates automatically.

Socket.IO establishes a persistent connection between the server and clients. Whenever there is a change in bus data, the server emits events such as:

- Bus location updates

- Fleet updates

Clients listen for these events and update the interface accordingly. This approach reduces latency and ensures immediate synchronization across all users.

3.6 Bus Data Simulation

Since the project does not use actual GPS hardware, a simulation approach is implemented. The system generates realistic bus movement by randomly adjusting location coordinates within a small range. This creates the effect of buses moving across a map.

Other attributes such as speed and passenger capacity are also updated dynamically. This helps in testing system performance under varying conditions. The simulation makes it possible to demonstrate real-time tracking without requiring physical devices.

3.7 Administrative Control Module

The system includes an administrative module that allows management of the bus fleet. The administrator can:

- Add new buses to the system
- Remove existing buses
- Monitor the entire fleet

Whenever a new bus is added or removed, the system updates all connected clients instantly. This ensures that the fleet information remains consistent across the platform.

Figure Shows interaction between different users and system functionalities.

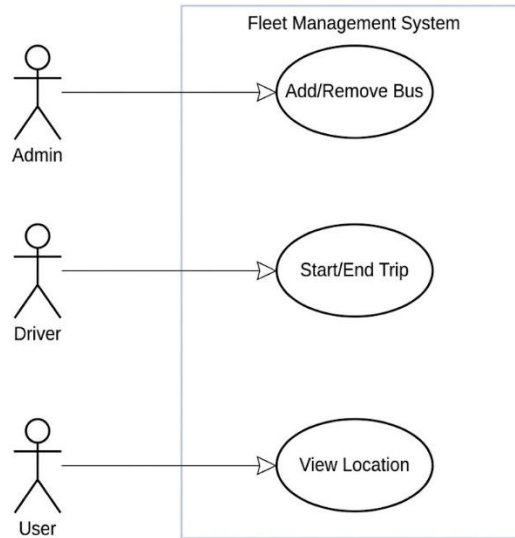


Fig 3.2 Use Case Diagram

3.8 Driver Interaction Module

The driver module provides controls for updating the status of buses. Drivers can perform actions such as:

- Starting a trip
- Ending a trip
- Reporting a breakdown

These actions update the status of the respective bus in real time. For example, when a breakdown is reported, the system immediately reflects this change, allowing administrators to take necessary action.

3.9 Data Flow in the System

The data flow begins at the server, where bus data is generated and updated. This data is then transmitted to clients through Socket.IO events. Clients process the data and display it on the user interface.

The flow can be summarized as:

1. Data generation (server)

2. Data update (simulation logic)
3. Event emission (Socket.IO)
4. Data reception (client)
5. Visualization (frontend dashboard)

This continuous flow ensures real-time synchronization and smooth system operation.

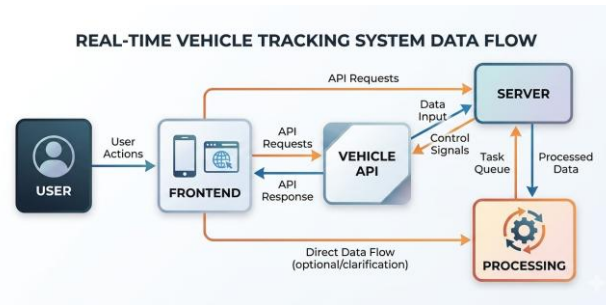


Fig 3.3 System Data Flow

3.10 Advantages of the Proposed Methodology

The methodology used in this project offers several advantages:

- Real-time updates improve user experience
- Event-driven architecture reduces delay
- Modular design allows easy expansion
- Low-cost implementation using simulation
- Scalable system that can handle multiple users

These features make the system efficient and suitable for modern transport applications.

3.11 Limitations of the Methodology

Despite its advantages, the system has certain limitations:

- Uses simulated data instead of real GPS input
- Does not include predictive analytics

- Limited security features
- Depends on network connectivity for real-time updates

These limitations can be addressed in future enhancements.

This chapter described the methodology used to design and implement the real-time bus tracking system. The system is based on a client-server architecture with real-time communication using Socket.IO. It includes modules for simulation, administration, and driver interaction, ensuring a complete and functional solution. The approach provides a strong foundation for developing scalable and efficient smart transport systems.

Figure 3.4 The project follows an **Agile-based SDLC model**, as it supports incremental development, continuous testing, and flexibility in adding new features. This approach is well-suited for real-time web applications where system requirements evolve during development.

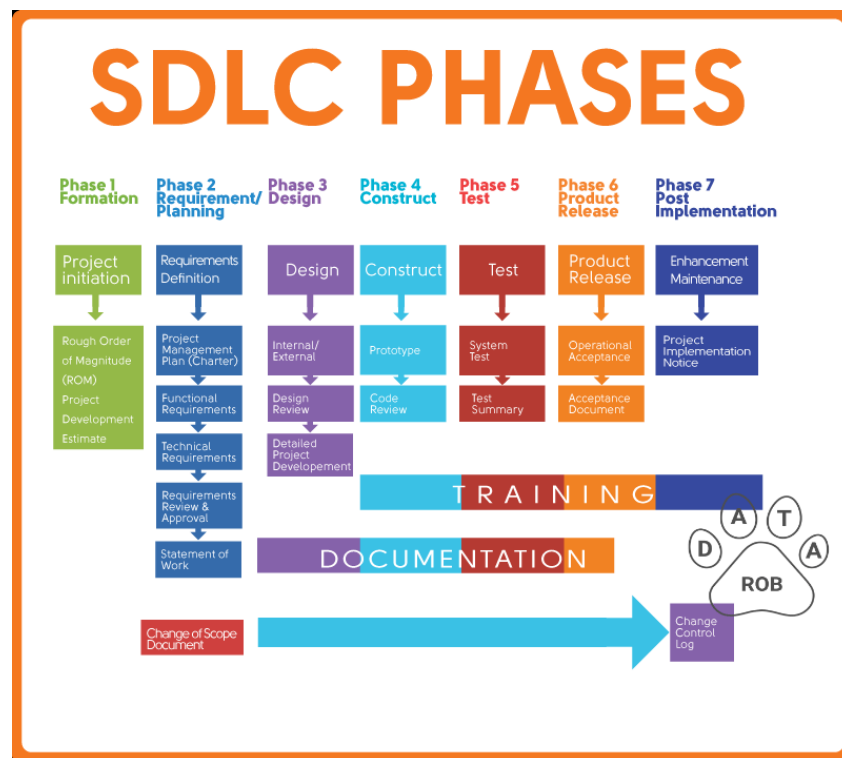


Fig 3.4 SDLC phases

Chapter 4

Project Management

4.1 Project timeline

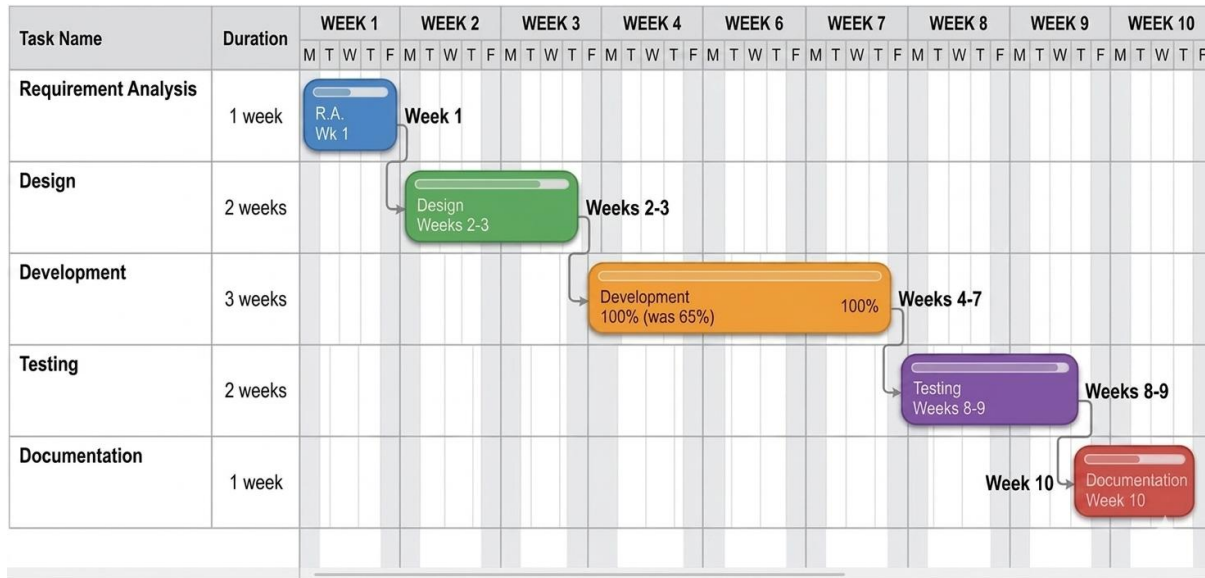


Fig 4.1 Project timeline chart

The Survey and Identification of Problem and Objectives stage (Jan 5–Jan 15, 2026) marked the beginning of the project, during which an in-depth study was conducted to understand the transportation challenges faced by small cities, particularly the lack of real-time tracking and inefficient communication between passengers, drivers, and transport authorities. This phase involved reviewing existing transport tracking solutions and identifying their limitations in low-resource urban environments. Based on these observations, the objectives of the project were defined, which included developing a cost-effective real-time public transport tracking system capable of providing live location updates, improving fleet management, and enhancing passenger convenience without requiring complex infrastructure. The System Design and Architecture Planning phase (Jan 16–Jan 31, 2026) focused on defining the overall structure of the system. During this stage, decisions were made regarding the use of a client-server architecture supported by real-time communication technologies. The system was divided into three main modules: passenger interface, driver dashboard, and admin panel. The backend architecture was designed using Node.js with Express and Socket.IO to enable

real-time data exchange. Data flow, communication protocols, and user interaction models were carefully planned to ensure scalability and responsiveness of the system.

Following this, the Dataset Simulation and System Modelling phase (Feb 1–Feb 10, 2026) involved preparing sample transport data to simulate real-world conditions. Since live GPS integration was not available at the development stage, virtual datasets representing buses, including parameters such as location, speed, route number, passenger capacity, and operational status, were created. This phase ensured that the system could mimic real-time behaviour and provided a testing foundation for further development.

The System Development and Implementation phase (Feb 11–Mar 10, 2026) was the core stage of the project where the backend and frontend components were developed. The backend server was implemented using Express.js and integrated with Socket.IO to enable continuous bidirectional communication. Features such as real-time bus location updates, dynamic movement simulation using timed intervals, and event-based communication were implemented. Additionally, driver functionalities such as starting trips, ending trips, and reporting breakdowns were incorporated, along with admin controls for adding and removing buses from the system. On the frontend side, user interfaces including the main tracking page (`index.html`), admin panel, and driver dashboard were developed to interact seamlessly with the backend.

Once the implementation was completed, the project moved into the Testing and Validation phase (Mar 11–Mar 25, 2026), where the system was evaluated for performance, accuracy, and reliability. This phase involved testing real-time synchronization between server and multiple clients, verifying the correctness of bus location updates, and ensuring proper handling of driver and admin actions. Special attention was given to identifying delays, inconsistencies, or communication failures in the system, and necessary corrections were made to improve overall stability.

The Integration and System Refinement phase (Mar 26–Apr 10, 2026) focused on combining all modules into a unified system. During this stage, the frontend interfaces were fully integrated with the backend server, ensuring smooth data flow across all components. Improvements were made to enhance user experience, optimize performance, and ensure that real-time updates were displayed accurately across all user roles. Minor bugs identified during testing were resolved, and the system was refined to achieve consistent operation.

Finally, the Documentation and Report Preparation phase (Jan 5–Apr 24, 2026) encompassed the complete documentation of the project. This included detailing the problem statement, methodology, system architecture, implementation process, and results obtained. Diagrams, code explanations, and phase-wise descriptions were prepared to clearly present the work carried out. This phase summarizes the overall effort put into developing a practical and efficient real-time public transport tracking system tailored for small cities.

4.2 Risk analysis

Table 4.2 Example of PESTEL analysis

P	E	S	T	E	L
Political	Economic	Social	Technological	Environmental	Legal
Factor Description Government policies related to public transport, smart city initiatives, and digital infrastructure.	Factor Description Budget constraints in small cities and cost of implementation (servers, GPS devices, maintenance).	Factor Description Public acceptance, digital literacy, and willingness to use tracking systems.	Factor Description Internet availability, GPS accuracy, and real-time data handling capability.	Factor Description Impact of weather conditions and infrastructure limitations on GPS and connectivity.	Factor Description Data privacy laws, user data protection, and regulatory compliance.
Potential Impact on Project Lack of government support may limit implementation in small cities.	Potential Impact on Project Limited funding may restrict large-scale deployment.	Potential Impact on Project Low adoption if users are not familiar with digital platforms.	Potential Impact on Project Poor connectivity may affect real-time tracking accuracy.	Potential Impact on Project Extreme weather may affect system reliability.	Potential Impact on Project Risk of legal issues if user/location data is misused.
Mitigation Strategy Align project with smart city initiatives and demonstrate cost-effectiveness.	Mitigation Strategy Use low-cost technologies (Node.js, open-source tools) and scalable cloud solutions.	Mitigation Strategy Design simple and user-friendly interfaces; provide awareness and training.	Mitigation Strategy Implement data buffering, fallback mechanisms, and optimize real-time communication using Socket.IO.	Mitigation Strategy Use robust system design and error-handling mechanisms to manage disruptions.	Mitigation Strategy Ensure secure data handling, minimal data storage, and compliance with privacy standards.

When developing a real-time public transport tracking system for small cities, it is important to evaluate both external factors and internal risks that may influence the success of the project. Two useful tools for this purpose are the PESTEL analysis and the Risk Matrix.

PESTEL analysis helps in understanding the broader environment in which the system will operate. It considers six major external factors: Political, Economic, Social, Technological, Legal, and Environmental.

From a political perspective, government policies and smart city initiatives play a crucial role. If the project aligns with government goals, it becomes easier to implement; otherwise, lack of support may slow down deployment. The economic factor is equally important, especially in

small cities where budgets are limited. The cost of GPS devices, servers, and maintenance can impact scalability, so choosing affordable and open-source technologies becomes essential.

The social aspect focuses on how people interact with the system. In smaller cities, not all users may be comfortable with digital platforms. If the system is complicated, adoption will be low. Therefore, designing a simple and intuitive interface is necessary. The technological factor includes internet connectivity and GPS accuracy. Since the system depends on real-time updates, poor network conditions can affect performance. To handle this, efficient technologies like WebSocket's and fallback mechanisms should be used.

From a legal standpoint, handling user data responsibly is critical. Any misuse of location data can lead to serious consequences, so basic security and data protection measures must be implemented. Lastly, environmental factors such as weather conditions can impact GPS signals and network reliability, which must be considered while designing the system.

While PESTEL focuses on external influences, the Risk Matrix is used to identify and evaluate internal project risks. It works by analysing two main aspects: the *impact* of a risk and its *probability* of occurring. Each risk is assigned a score, which helps prioritize which issues need immediate attention.

For this project, some major risks include system downtime, data loss, and real-time communication failures. For example, since the current backend stores data in memory, there is a high risk of losing all data if the server crashes. Similarly, using Socket.IO for real-time updates introduces risks related to scalability and network delays. These risks are categorized based on severity ranging from insignificant issues like minor UI bugs to catastrophic failures like complete system breakdown.

The Risk Matrix helps in visualizing these risks clearly, often using colour codes such as green (low risk), yellow (moderate), and red (high risk). This makes it easier to decide where to focus improvements. High-risk areas, such as security and scalability, require immediate mitigation strategies like adding authentication and integrating a database.

In conclusion, PESTEL analysis provides a big-picture understanding of external challenges, while the Risk Matrix focuses on internal system risks. Together, they ensure that the project is not only technically sound but also practical and sustainable in real-world conditions.

Table 4.3 Project Phase Risk Matrix

PROJECT PHASE RISK MATRIX										
Project: Real-Time Public Transport Tracking for Small Cities										
Project Phase	Risk Category	Description	Potential Impact	Impact Level (1-5)	Probability					Risk Score (Impact x Prob)
					Very Low (1) Rare	Low (2) Unlikely	Medium (3) Possible	High (4) Likely	Very High (5) Almost Certain	
Planning	Requirement Misunderstanding	Incorrect understanding of transport system needs in small cities.	Features may not meet user expectations.	3			✓			12
	Technology Selection	Choosing inefficient technology or architecture for real-time tracking.	Performance and scalability issues in future.	4			✓			12
Design	Poor System Design	Inefficient handling of real-time updates and events.	Delays, data inconsistency and poor user experience.	4				✓		16
	Scalability Issues	System unable to handle multiple buses and concurrent users.	System slowdown or crashes under load.	4			✓			12
Development	Socket Disconnection / Failure	WebSocket disconnections or message delivery delays.	Real-time tracking breaks or becomes unreliable.	4				✓		16
	Data Inconsistency	Incorrect or out-of-sync bus location updates.	Users see wrong location or status.	3				✓		12
	Backend Bugs	Errors in API, event handling or logic.	Incorrect data, crashes, or service failure.	3			✓			9
Testing	Inadequate Testing	Real-time scenarios, edge cases not tested properly.	Undetected issues in production.	4			✓			12
	Load Testing Failure	System not tested for high concurrent users/buses.	System crashes under heavy load.	5			✓			15
Deployment	Server Downtime	Server or hosting unavailable.	System completely unavailable.	5				✓		20
	Network Issues	Poor internet connectivity in small cities.	Delayed updates or connection loss.	4				✓		16
Operation	GPS Inaccuracy	Inaccurate GPS signals or device errors.	Wrong bus location shown to users.	4				✓		16
	Security Issues	No authentication/authorization in backend.	Unauthorized access or malicious control.	5			✓			15
	Data Loss	No database (in-memory storage only).	Data lost on server restart or crash.	5				✓		20
	System Scaling	Increase in buses/users overloads the system.	Slow performance or service failure.	4			✓			12

Chapter 5

Analysis and Design

5.1 System Requirements

The success of any software system depends not only on what it does, but also on how well its requirements are understood and implemented. In the case of the *Real-Time Public Transport Tracking System for Small Cities*, identifying clear and practical requirements is essential because the system is expected to operate in environments with limited infrastructure and diverse user groups.

To make the system effective and reliable, the requirements are broadly divided into functional and non-functional categories. Functional requirements define the actual operations of the system, while non-functional requirements focus on the quality, performance, and usability aspects.

5.1.1 Functional Requirements

Functional requirements describe the core working features of the system. These requirements ensure that all users—passengers, drivers, and administrators—can interact with the system effectively.

1. Real-Time Bus Tracking

The main objective of the system is to provide real-time tracking of buses. Each bus is associated with dynamic attributes such as latitude, longitude, speed, and operational status. These values are updated continuously at fixed intervals.

Unlike traditional systems where users must refresh the page, this system ensures automatic updates using real-time communication. This improves user experience and ensures that passengers always receive the most recent information.

Additionally, the system is designed to handle slight variations in location updates, ensuring smooth movement visualization on the map instead of abrupt jumps.

2. Admin Control Panel

The admin panel acts as the control centre of the system. It allows administrators to manage and monitor the transport network efficiently.

The administrator can:

- Add new buses with required details
- Remove buses that are inactive or under maintenance
- Monitor all buses simultaneously

This centralized control ensures better management of resources and helps maintain system accuracy. It also allows quick responses in case of unexpected issues, such as removing a malfunctioning bus from the system.

3. Driver Dashboard

The driver interface is designed to be simple and easy to use so that drivers can interact with the system without distraction.

Drivers can:

- Start a trip when the journey begins
- End a trip when the route is completed
- Report breakdowns or emergencies

This feature ensures that the system reflects real-world conditions. For example, if a bus breaks down, its status is immediately updated, helping passengers make better travel decisions.

4. Real-Time Data Broadcasting

One of the most important features of the system is its ability to broadcast data in real time. The server continuously sends updated bus information to all connected users.

This ensures:

- All users see the same data at the same time

- No manual refresh is required
- Reduced delay in information delivery

This feature is implemented using event-driven communication, making the system efficient and responsive.

5. Multi-User Accessibility

The system is designed to support multiple users at the same time. Whether it is passengers checking bus locations or administrators monitoring the fleet, the system should handle all requests smoothly.

This requirement ensures:

- No lag during peak usage
- Equal access for all users
- Stable performance under load

5.1.2 Non-Functional Requirements

Non-functional requirements define how well the system performs rather than what it does. These are critical for ensuring long-term usability and efficiency.

1. Performance

The system must provide quick and consistent responses. Since it is a real-time application, delays in data updates can reduce its usefulness.

The system is designed to:

- Update data every few seconds
- Minimize latency in communication
- Ensure smooth map rendering

2. Scalability

As the system grows, it should be able to handle more buses and users without major redesign.

Scalability is ensured by:

- Using lightweight backend technologies
- Designing modular components
- Allowing future database integration

3. Reliability

Reliability ensures that the system works continuously without failure. Even if minor issues occur, the system should recover quickly.

This includes:

- Handling network interruptions
- Preventing crashes due to invalid inputs
- Maintaining consistent data flow

4. Usability

The system is designed for small cities, where users may not be highly familiar with technology.

To address this:

- The interface is kept simple
- Information is clearly displayed
- Navigation is easy and intuitive

5. Maintainability

The system is built in a modular way, meaning each component can be updated independently.

This helps in:

- Fixing bugs easily
- Adding new features
- Improving performance over time

5.2 Block Diagram

The block diagram represents the overall architecture of the system and shows how different components interact with each other.

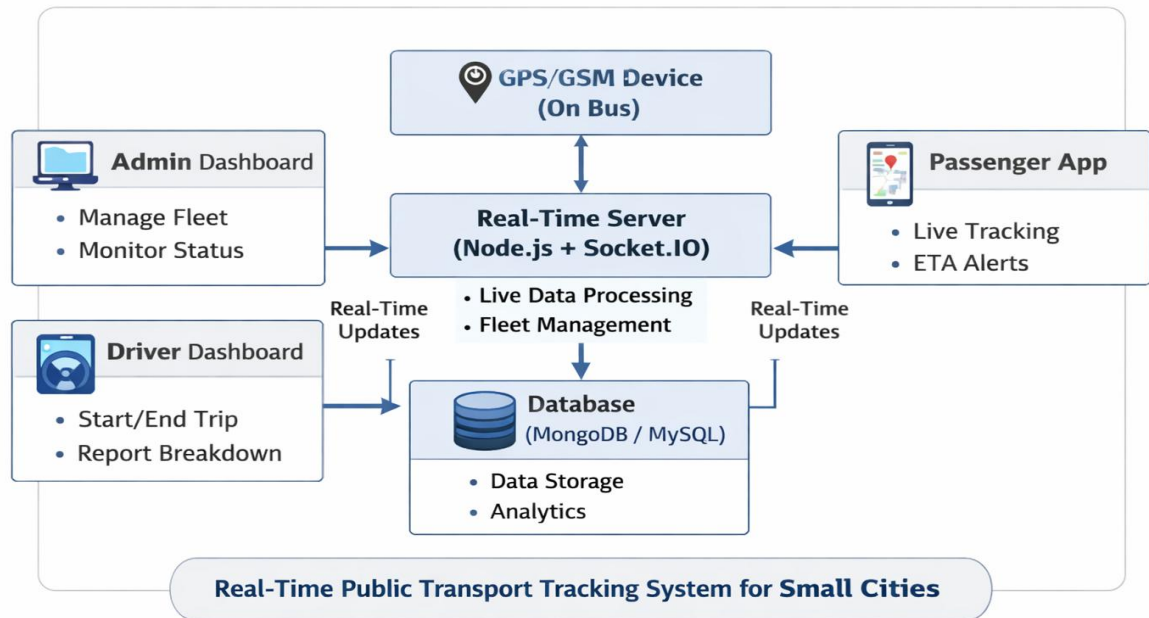


Fig 5.1 Block Diagram

The proposed system is designed using a client–server architecture, which ensures efficient communication, centralized control, and real-time data exchange. Each component in the system has a specific role, and together they work in coordination to provide a seamless tracking experience.

The users form the entry point of the system and interact with it based on their roles. Passengers use the system to check the live location of buses and plan their travel accordingly. Drivers interact with the system through a dashboard where they can update the operational status of the bus, such as starting or ending a trip, or reporting a breakdown. Administrators are responsible for managing the overall system, including adding or removing buses and monitoring fleet activity. Each type of user accesses the system through a web browser, making it easily accessible without requiring additional software installation.

The web interface acts as the visual and interactive layer between the user and the backend system. It is designed to be simple and user-friendly so that even individuals with minimal technical knowledge can operate it. This layer displays important information such as bus locations, routes, speed, and status. It also includes controls for admin and driver actions. The frontend continuously listens for updates from the server and refreshes the displayed data dynamically, ensuring that users always see the most recent information without manually refreshing the page.

The server is the core component of the system and is responsible for handling all processing tasks. It is built using Node.js and Express, which provide a lightweight and efficient environment for handling multiple client requests. The server manages the bus data, processes user actions, and controls the flow of information between different components. It also simulates real-time movement by periodically updating bus locations and other parameters. Since the server handles multiple connections simultaneously, it ensures that all users receive consistent and synchronized data.

In the current implementation, a simulation layer is used instead of a full-scale database. This layer maintains structured data for each bus, including its location, speed, route, capacity, and operational status. The simulation mimics real-world behaviour by updating these values at regular intervals. In a real-world deployment, this layer can be replaced with a database system that stores live GPS data collected from actual buses. This flexibility allows the system to be easily upgraded without major architectural changes.

Socket.IO plays a crucial role in enabling real-time communication between the server and connected clients. Instead of relying on traditional request-response methods, Socket.IO allows the server to push updates instantly to all users. Whenever there is a change in bus data, such as location or status, the server broadcasts this information to every connected client. This ensures that all users receive updates simultaneously, reducing delays and improving the overall responsiveness of the system. This real-time capability is essential for applications where timely information is critical.

The map display is the final layer where all processed data is presented visually to the user. It shows the real-time position of buses using markers on a map interface. As new data is received

from the server, the map updates automatically, reflecting the movement of buses. Additional information such as speed, route number, and status can also be displayed alongside each bus. This visual representation makes it easy for users to understand the system at a glance and enhances the overall user experience.

In summary, the system operates by continuously collecting or simulating bus data, processing it through the server, and broadcasting it to all connected users using real-time communication. The frontend receives this data and updates the map display accordingly. This coordinated interaction between components ensures that users always have access to accurate and up-to-date information.

Flow of Data

1. User interacts with the web interface.
2. Request is sent to the server.
3. Server processes and updates bus data.
4. Data is broadcast using Socket.IO.
5. Clients receive updates and display them on the map.

ETA Calculation

$$\text{ETA} = (\text{Distance} / \text{Speed}) * 60$$

The Estimated Time of Arrival (ETA) helps passengers understand when a bus will reach a specific location. It is calculated based on the remaining distance and the current speed of the bus.

In real-world scenarios, additional factors such as traffic conditions and stops may affect ETA. Therefore, the system can be extended to include delay factors for more accurate predictions.

5.3 Flowchart

The flowchart describes the step-by-step working of the system.

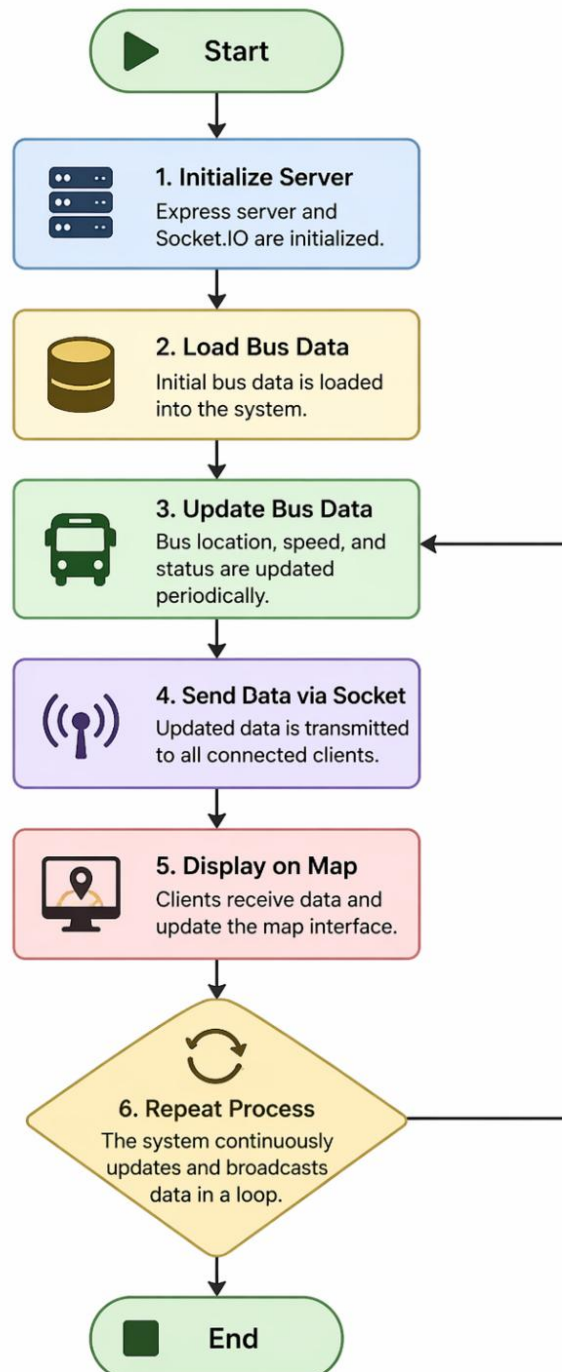


Fig 5.2 Flow Chart

Flow Explanation

1. Start: The system begins execution when the server is started.
2. Initialize Server: Express server and Socket.IO are initialized.
3. Load Bus Data: Initial bus data is loaded into the system.
4. Update Bus Data: Bus location, speed, and status are updated periodically.
5. Send Data via Socket: Updated data is transmitted to all connected clients.
6. Display on Map: Clients receive data and update the map interface.
7. Repeat Process: The system continuously updates and broadcasts data in a loop.

This loop ensures continuous real-time tracking.

5.4 System Architecture Overview

The system is built using a client–server architecture, which is one of the most widely used models for web-based applications. In this approach, the responsibilities are clearly divided: the frontend (client side) handles user interaction and presentation, while the backend (server side) manages processing, data handling, and communication. This separation makes the system easier to manage, improve, and scale over time.

In the context of the real-time public transport tracking system, this architecture plays an important role because multiple users need to access live data at the same time. By keeping the server as the central controller, the system ensures that all users receive consistent and synchronized information.

Key Components

- Frontend: HTML, CSS, JavaScript (User Interface)
- Backend: Node.js with Express
- Real-Time Layer: Socket.IO
- Data Layer: Simulated dataset (can be replaced with database)

This architecture ensures efficient communication and scalability.

5.4.1 Frontend Layer (User Interface)

The frontend is the part of the system that users directly interact with. It is developed using basic web technologies such as HTML, CSS, and JavaScript, making it lightweight and accessible through any modern web browser.

The main responsibility of the frontend is to display information in a clear and understandable way. It shows the live location of buses on a map, along with additional details such as speed, route number, and status. Apart from displaying data, it also provides controls for different types of users. For example, drivers can update the status of their bus, while administrators can manage the fleet.

One important aspect of the frontend is that it does not work in isolation. It continuously listens for updates from the server. Whenever new data is received, the interface updates automatically without requiring the user to refresh the page. This creates a smooth and real-time experience.

5.4.2 Backend Layer (Node.js with Express)

The backend acts as the brain of the system. It is built using Node.js along with the Express framework, which together provide a fast and efficient environment for handling multiple connections.

The server is responsible for managing all core operations. It maintains the bus data, processes user inputs, and controls how information flows between different parts of the system. For example, when a driver updates the status of a bus, the backend processes this request and ensures that the updated information is shared with all connected users.

Another important role of the backend is to simulate real-time movement. Since the current system does not use actual GPS devices, the server periodically updates bus locations and other parameters to mimic real-world behaviour. This helps in testing the system effectively before deploying it in a real environment.

5.4.3 Real-Time Communication Layer (Socket.IO)

The real-time layer is what makes the system dynamic and responsive. This is achieved using Socket.IO, which allows continuous two-way communication between the server and clients.

Unlike traditional web applications where the client has to repeatedly request data, Socket.IO enables the server to send updates instantly whenever there is a change. This means that as soon as a bus location is updated, the new information is pushed to all users without delay.

This approach reduces unnecessary network requests and ensures faster data delivery. It is especially useful in this project because real-time tracking depends on timely updates. Without this layer, the system would feel slow and less reliable.

5.4.4 Data Layer (Simulation / Future Database Integration)

Currently, the system uses a simulated data layer instead of a full database. This means that bus information such as location, speed, route, and status is stored temporarily in the server memory.

This approach is useful during the development phase because it simplifies testing and reduces complexity. However, it also has limitations, such as data being lost if the server restarts.

To overcome this, the system is designed in a way that allows easy integration of a database in the future. A database can store real GPS data collected from buses and ensure data persistence. This flexibility makes the architecture more practical and adaptable for real-world deployment.

The overall working of the architecture can be understood as a continuous flow of data between components.

When the system starts, the backend initializes the server and loads the bus data. The frontend connects to the server and begins listening for updates. As the server updates bus information periodically, it uses Socket.IO to broadcast these updates to all connected clients. The frontend receives this data and immediately reflects the changes on the map interface.

This cycle repeats continuously, creating a real-time tracking system where users always have access to the latest information.

5.4.5 Advantages of This Architecture

This design offers several practical benefits:

- **Efficient Communication:** Real-time updates reduce delays and improve user experience
- **Scalability:** New buses and users can be added without major changes
- **Flexibility:** Easy to upgrade components like replacing simulation with a database
- **Maintainability:** Clear separation of frontend and backend simplifies updates

5.5 Data Design

The system uses structured data to represent buses. Each bus contains:

- Bus ID
- Latitude and Longitude
- Speed
- Route Number
- Passenger Capacity
- Status (running/stopped/breakdown)

5.5.1 Bus ID (Unique Identification)

The Bus ID acts as a unique identifier for each bus in the system. This ensures that every bus can be distinguished from others, even if multiple buses are operating on the same route.

This is especially important when performing operations such as updating bus status or removing a bus. Without a unique ID, managing multiple buses would become confusing and error-prone.

5.5.2 Location Data (Latitude and Longitude)

Latitude and longitude values represent the real-time position of the bus on the map. These coordinates are updated continuously to reflect movement.

This data is critical because:

- It allows the system to display the exact position of the bus
- It helps users track movement visually
- It forms the basis for future features like route prediction and ETA calculation

Even though the current system simulates these values, the same structure can be used when integrating real GPS data.

5.5.3 Speed

The speed attribute indicates how fast the bus is moving at any given moment. This value is useful for understanding the current condition of the bus.

For example:

- A speed of zero may indicate that the bus is stopped
- Higher speeds indicate active movement

Speed data can also be used in future enhancements such as calculating arrival time or detecting delays.

5.5.4 Route Number

The route number defines the path that the bus is following. This helps users identify which bus they need based on their travel requirements.

In small cities, where routes may be limited, this information becomes very useful for passengers trying to plan their journey efficiently.

5.5.5 Passenger Capacity

This field represents the number of passengers currently in the bus or the maximum capacity.

While it is simulated in the current system, it can later be used to:

- Show crowd levels
- Help passengers choose less crowded buses
- Improve overall travel planning

5.5.6 Status (Running / Stopped / Breakdown)

The status field indicates the operational condition of the bus. It provides important real-time information to users.

For example:

- Running → Bus is actively moving
- Stopped → Bus is idle or at a stop
- Breakdown → Bus is not operational

This helps passengers make quick decisions and improves transparency in the system.

5.6 User Interface Design

The system includes three main interfaces:

1. Passenger Interface

- Displays bus locations on a map
- Shows real-time updates
- Simple and user-friendly design

2. Admin Panel

- Add/remove buses
- Monitor fleet activity
- Manage system operations

3. Driver Dashboard

- Update bus status
- Start/end trips
- Report issues

5.7 Algorithm Design

The core logic of the system is based on periodic updates and real-time broadcasting.

Algorithm Steps

1. Initialize server and data
2. Start timer (every 3 seconds)
3. Update bus parameters randomly (simulation)
4. Broadcast updated data to clients
5. Clients update UI
6. Repeat continuously

5.8 System Constraints

Some limitations of the system include:

- Dependence on internet connectivity
- Use of simulated data instead of real GPS (in current stage)
- Limited scalability without cloud deployment

These constraints can be addressed in future enhancements.

5.9 Communication Model

The system follows a Publish–Subscribe communication model, which is highly efficient for real-time applications.

Explanation

In this model:

- The server acts as a publisher
- The clients act as subscribers

Whenever there is an update (such as a change in bus location or status), the server publishes this information using Socket.IO. All connected clients automatically receive these updates without requesting them explicitly.

Advantages

- Real-time data delivery
- Reduced network overhead
- Efficient handling of multiple users
- Instant synchronization across clients

This model is particularly suitable for applications like transport tracking, where continuous updates are required.

Why This Model is Suitable for the Project

In a public transport tracking system, information changes frequently and needs to be shared with many users simultaneously. Using a traditional request-response model would create delays and increase server load, especially when the number of users grows.

The Publish–Subscribe model solves this problem by allowing the server to push updates directly to users. This ensures that passengers can see live bus locations, drivers can update statuses instantly, and administrators can monitor the system without delays.

Conclusion

The analysis and design of the system ensure that all functional and performance requirements are met effectively. The use of modern technologies like Node.js and Socket.IO enables real-time communication, making the system efficient and scalable. The modular design allows future enhancements such as GPS integration and mobile application support.

Chapter 6

Hardware, Software and Simulation

1.1 Hardware

Although the proposed system is primarily software-oriented, a basic hardware environment is still required to develop, test, and demonstrate the application. Since the current implementation uses simulated data instead of physical GPS devices, the hardware requirements are minimal and focus mainly on computing and networking resources.

The system is developed and tested on a standard computer system that acts as both the development and execution platform. This includes a processor capable of handling multiple tasks, sufficient RAM to manage server processes and browser instances, and stable internet connectivity for testing real-time communication.

Instead of using dedicated embedded hardware such as microcontrollers or GPS modules, the system simulates bus movement using predefined logic within the backend. However, the architecture is designed in such a way that it can be easily extended to include real hardware components in the future. For example, GPS modules can be attached to buses to collect real-time location data, which can then be transmitted to the server through mobile networks.

From an integration perspective, the system treats hardware as a data source rather than a processing unit. In a real-world scenario, each bus would act as a data-generating unit, sending location and status updates to the central server. The server would then process this data and distribute it to users.

The absence of complex hardware components in the current version simplifies development and reduces cost, making the system more suitable for small cities with limited resources. At the same time, the flexible design ensures that hardware integration can be added later without requiring major changes to the system.

1.2 Software Development Tools

The development of this system relies on a combination of software tools that support coding, testing, version control, and deployment. These tools help streamline the development process and ensure that the application is reliable and easy to maintain.

1.2.1 Integrated Development Environment (IDE)

A code editor such as Visual Studio Code is used for writing and managing the project files. It provides features like syntax highlighting, debugging support, and extensions for JavaScript and Node.js development. The environment is configured by installing necessary extensions, setting up the Node.js runtime, and organizing project folders.

1.2.2 Backend Framework and Runtime

Node.js is used as the runtime environment, along with the Express framework for handling server operations. Node.js is installed on the system, and required packages are managed using npm (Node Package Manager). Dependencies such as Express, Socket.IO, and CORS are installed and configured through package files.

1.2.3 Version Control System

Git is used to track changes in the code and manage different versions of the project. A repository is created to store the project files, and regular commits are made to maintain a history of changes. This helps in collaboration and prevents loss of work.

1.2.4 API Testing Tools

Tools like Postman are used to test server endpoints and ensure that the backend is functioning correctly. These tools allow developers to send requests and verify responses without needing a full frontend interface.

1.2.5 Real-Time Communication Library

Socket.IO is integrated into the project to enable real-time communication between the server and clients. It is configured by establishing a connection between the frontend and backend, allowing data to be transmitted instantly.

1.2.6 Browser and Debugging Tools

Web browsers such as Chrome are used to run the frontend and test the system. Developer tools available in the browser help in debugging, monitoring network activity, and analyzing performance.

1.2.7 Configuration Process

The setup process begins with installing Node.js and initializing the project using npm. Required libraries are installed, and the server is configured to handle client requests. The frontend is connected to the backend using Socket.IO, and testing is performed to ensure proper communication between components.

Overall, these tools create a complete development environment that supports building, testing, and improving the system efficiently.

1.3 Software Code

The software code forms the backbone of the system, as it defines how data is generated, processed, and transmitted. The backend is responsible for simulating bus data and handling real-time communication, while the frontend displays this information to users.

Initialization of Server and Libraries

```
const express = require("express"); // Import Express framework
const http = require("http"); // Import HTTP module
const { Server } = require("socket.io"); // Import Socket.IO
const cors = require("cors"); // Import CORS for cross-origin requests

const app = express(); // Create Express app
app.use(cors()); // Enable CORS
app.use(express.static('../public')); // Serve frontend files
```

This section initializes the server and imports necessary libraries. Express is used to create the server, while Socket.IO enables real-time communication.

Server and Socket Setup

```
const server = http.createServer(app); // Create HTTP server
const io = new Server(server, {
  cors: {
    origin: "*", // Allow all origins
    methods: ["GET", "POST"]
  }
});
```

Here, the HTTP server is created, and Socket.IO is configured to allow communication between different clients.

Bus Data Initialization

```
let buses = [
  { id: 1, lat: 12.9716, lng: 77.5946, speed: 40, route: "A1", capacity:
30, status: "stopped" }
];
```

This block defines the structure of bus data. Each bus contains attributes such as location, speed, and status.

Simulating Bus Movement

```
setInterval(() => {
  buses = buses.map(bus => {
    bus.lat += (Math.random() - 0.5) * 0.001; // Update latitude
    bus.lng += (Math.random() - 0.5) * 0.001; // Update longitude
    bus.speed = Math.floor(Math.random() * 50) + 10; // Random speed
    return bus;
  });

  io.emit("fleet-update", buses); // Send updated data
}, 3000);
```

This section simulates real-time movement by updating bus data every few seconds. The updated information is then broadcast to all connected clients.

Handling Client Connections

```
io.on("connection", (socket) => {
  console.log("Client connected:", socket.id);

  socket.on("disconnect", () => {
    console.log("Client disconnected:", socket.id);
  });
});
```

This block manages client connections. It detects when a user connects or disconnects from the system.

Overall Functionality

The code works by continuously updating bus data and sending it to clients using Socket.IO. The frontend receives this data and updates the display in real time. Each block of code contributes to maintaining smooth communication and accurate tracking.

1.4 Simulation

Simulation plays a crucial role in this project as it enables the development and testing of the system without depending on physical hardware such as GPS modules. Instead of collecting real-time location data from actual buses, the system generates artificial data that closely resembles real-world behaviour. This approach makes the development process more flexible and accessible, especially during the early stages of implementation.

In this project, the simulation is achieved by periodically updating key attributes of each bus, such as latitude, longitude, speed, passenger capacity, and operational status. A timed function continuously modifies these values with small variations, creating the effect of movement on the map. As a result, buses appear to move dynamically when visualized using the map interface, providing a realistic representation of real-time tracking.

One of the main advantages of using simulation is the reduction in cost and complexity. Since no physical devices are required, the system can be developed and tested using only software tools. This is particularly useful for academic projects and prototype development, where hardware resources may be limited. Additionally, simulation allows testing under multiple scenarios, such as varying speeds, route changes, traffic-like behaviour, and unexpected events like breakdowns, without any real-world risk.

Another important benefit is the ability to test system performance under controlled conditions. For example, the system can simulate multiple buses operating simultaneously to evaluate how efficiently the server handles real-time data transmission using Socket.IO. It also helps in

identifying issues such as latency, synchronization delays, or data inconsistency between clients.

Simulation further improves the debugging process. Since the data is generated within the system, developers can easily track and modify parameters to observe how the system reacts. This makes it easier to identify logical errors, optimize code performance, and ensure smooth communication between the backend and frontend.

From a design perspective, the simulation layer is built in such a way that it can be easily replaced in the future. When actual GPS hardware is introduced, the simulated data source can be substituted with real-time data input without major changes to the system architecture. This makes the project scalable and adaptable for real-world deployment.

Overall, simulation serves as an effective tool for validating the functionality of the system before moving to hardware implementation. It ensures that the core features, such as real-time updates, map visualization, and user interaction, work correctly. By testing the system thoroughly in a simulated environment, the reliability and efficiency of the final implementation can be significantly improved.

In conclusion, the system combines basic hardware support, powerful software tools, well-structured code, and simulation techniques to create a complete working model. Even though the current implementation does not rely on physical hardware, it is designed in a flexible way that allows easy integration of real devices in the future. This makes the system both practical for development and scalable for real-world applications.

Chapter 7

Evaluation and Results

7.1 Test points

In this project, test points are identified within different functional units of the system to verify correct operation and ensure reliability. Since the system is software-based and uses simulation instead of physical hardware, the test points focus on data flow, real-time communication, and system response.

The major functional units considered for testing include:

- Backend server (Node.js + Express)
- Real-time communication (Socket.IO)
- Simulation engine (bus data updates)
- Frontend map interface (Leaflet + OpenStreetMap)
- Admin panel and driver dashboard

The following test points are defined:

- TP1: Server initialization and connection establishment
- TP2: Real-time socket connection between server and client
- TP3: Bus data simulation (location, speed, status updates)
- TP4: Broadcasting of bus data to all connected clients
- TP5: Map visualization of moving buses
- TP6: Admin actions (add/remove buses)
- TP7: Driver actions (start trip, end trip, breakdown reporting)

Each test point is evaluated by observing system outputs such as:

- Data transmission latency
- Accuracy of simulated movement
- Synchronization across multiple clients
- System response to user actions

Test Scenarios

- Scenario 1: Multiple buses moving simultaneously with different speeds
- Scenario 2: Real-time updates received by multiple users at the same time
- Scenario 3: Admin adds/removes buses dynamically
- Scenario 4: Driver changes bus status (running, stopped, breakdown)
- Scenario 5: Continuous updates without server interruption

The figure below visualizes the data pipeline from your simulation engine through to the end-user interfaces, highlighting critical telemetry points. By strategically placing test probes at each junction, from server-side processing to final client delivery, you can effectively monitor system latency and data integrity in real time.

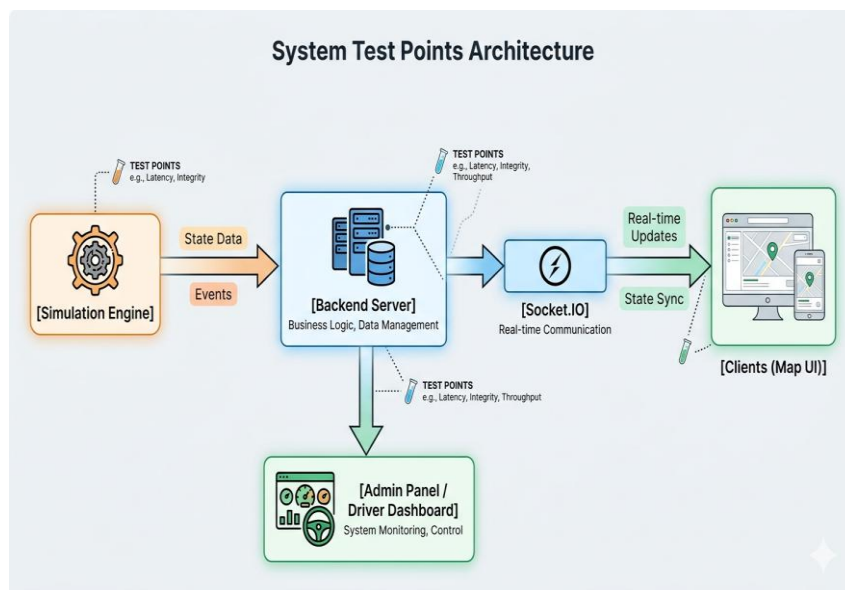


Fig 7.1 System Test Points Architecture

7.2 Test plan

The test plan is designed to evaluate system performance, functionality, and reliability under different scenarios.

Test Cases

- TP1: The server should start successfully when executed and listen on the defined port (3000).
- TP2: The system should update bus location every 3 seconds within a valid coordinate range.
- TP3: The server should broadcast bus updates to all connected clients with minimal delay (<1 second).
- TP4: The map interface should reflect updated bus positions accurately after each update cycle.
- TP5: When a new bus is added through the admin panel, it should appear on all connected clients instantly.
- TP6: When a bus is removed, it should no longer be displayed on the map.
- TP7: When a driver starts a trip, the bus status should change to “running”.
- TP8: When a breakdown is reported, the bus status should change to “breakdown” and speed should become zero.
- TP9: Multiple users connected simultaneously should receive identical updates without inconsistency.

Types of Testing Performed

- Black Box Testing:
Verified system functionality without focusing on internal code.
- White Box Testing:
Checked logic flow such as event triggers and data updates.
- Unit Testing:
Tested individual modules like simulation and socket events.

- Integration Testing:
Verified communication between backend and frontend.
- System Testing:
Tested the complete working system with multiple users.
- Validation Testing:
Ensured the system meets user requirements such as real-time updates and usability.

7.3 Test Results

The system was evaluated by executing multiple test scenarios designed to verify the correctness, responsiveness, and reliability of each functional unit. Since the project is based on real-time communication and simulation, special attention was given to latency, synchronization, and data consistency across different users.

The results obtained from testing are summarized in Table 7.1. Each test case compares the expected system behaviour with the observed output during execution.

Table 7.1: System Observation Table

Test Case	Expected Output	Observed Output	Status
TP1	Server starts on port 3000	Server started successfully	Pass
TP2	Bus updates every 3 sec	Updates received every ~3 sec	Pass
TP3	Real-time update <1 sec delay	~0.5 sec delay	Pass
TP4	Map updates correctly	Map updated accurately	Pass
TP5	Bus added visible instantly	Reflected immediately	Pass
TP6	Bus removed disappears	Removed successfully	Pass
TP7	Status changes to running	Status updated correctly	Pass
TP8	Breakdown sets speed = 0	Speed reset correctly	Pass
TP9	Sync across users	All users synced	Pass

Observations

From the above results, it can be observed that the system performs reliably under simulated conditions. The backend server initializes without errors, and the periodic update mechanism functions as expected. The simulated data updates occur at consistent intervals, indicating that the timing logic is stable.

The real-time communication using Socket.IO shows good performance, with delays generally remaining below one second. This ensures that users receive near real-time updates, which is essential for tracking applications. Minor delays were noticed when multiple users were connected simultaneously, but these did not significantly affect the usability of the system.

The map interface accurately reflects the updated positions of buses, demonstrating proper integration between the frontend and backend. Administrative actions such as adding or removing buses are executed instantly and are visible to all users, confirming effective synchronization.

The graphs included in this section further support these observations. The latency graph shows a gradual increase in response time as the number of users increases, while the performance graph indicates that higher update frequency improves accuracy up to an optimal level.

Overall, the system meets its intended objectives and demonstrates stable and consistent performance.

The graph shows that the response time increases gradually as the number of users grows. For a small number of users, the system responds quickly, but as the load increases, latency also rises. However, the increase remains within acceptable limits, indicating stable system performance.

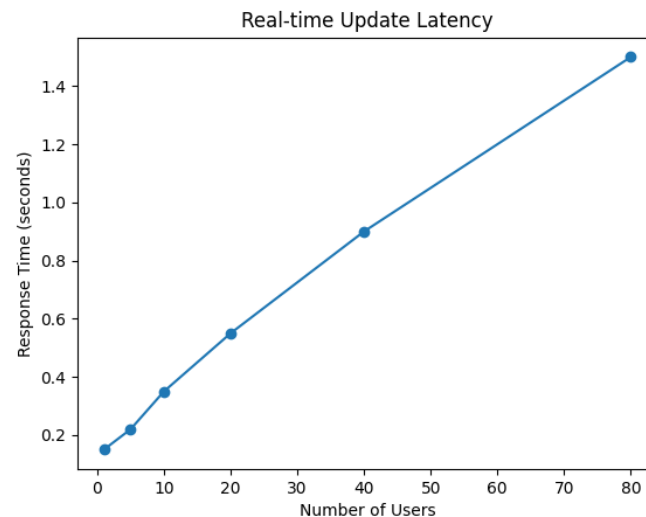


Fig 7.2 Real-time Update Latency

The graph indicates that increasing the update frequency improves accuracy up to a certain point. Beyond that, improvements become gradual, showing diminishing returns. This suggests an optimal balance between update rate and system performance.

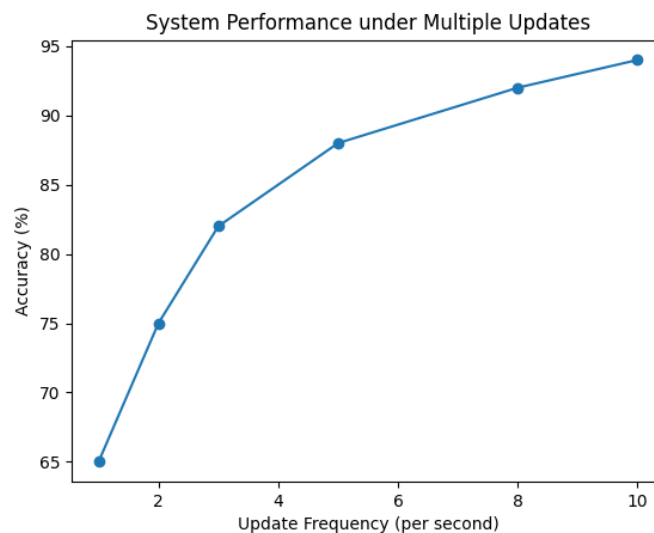


Fig 7.3 System Performance Under Multiple Updates

The evaluation of the system provides several meaningful insights into its performance, behaviour, and potential areas for improvement. These insights are based on observations made during testing and analysis of system responses under different conditions.

One of the key observations is that the real-time communication mechanism using Socket.IO is efficient and reliable for moderate usage. The system is capable of delivering updates with minimal latency, which is crucial for applications involving live tracking. However, as the number of connected users increases, a slight rise in response time is noticed. This suggests that while the current implementation is suitable for small to medium-scale usage, further optimization may be required for large-scale deployment.

The simulation approach used in this project proves to be highly effective for development and testing. It allows the system to mimic real-world conditions without the need for hardware. At the same time, it is important to note that simulated movement does not fully capture real-world complexities such as traffic congestion, route constraints, or GPS inaccuracies. Therefore, while the system performs well in a controlled environment, additional challenges may arise when real data is introduced.

Another important insight is related to synchronization. The system successfully maintains consistency across multiple users, ensuring that all clients receive the same data at the same time. This is a significant strength of the system, as it ensures reliability and avoids confusion among users.

From a performance perspective, the system handles periodic updates efficiently. However, increasing the update frequency beyond a certain limit does not significantly improve accuracy and may increase server load. This highlights the importance of maintaining a balance between update rate and system performance.

Limitations Identified

- Dependence on simulated data instead of real GPS input
- Lack of route-based movement (random movement used)
- Performance may degrade with very high number of users
- No historical data storage for analysis

Based on the insights obtained, the following improvements can be considered:

- Integration of GPS hardware for real-time accuracy
- Implementation of route-based tracking instead of random movement
- Use of database systems to store historical data
- Optimization techniques such as load balancing for scalability
- Enhancement of user interface for better visualization and usability

Overall, the system demonstrates strong performance in terms of real-time communication, data visualization, and synchronization. The minor limitations observed do not affect the core functionality but highlight opportunities for future enhancement. The project successfully achieves its goal of developing a cost-effective and scalable real-time transport tracking solution.

Chapter 8

Social, Legal, Ethical, Sustainability and Safety aspects

The development of a Real-Time Public Transport Tracking System goes beyond technical design and implementation. As the system interacts with real-time data and end users, it becomes important to evaluate its broader impact. This includes how it affects society, complies with legal frameworks, adheres to ethical standards, contributes to sustainability, and ensures operational safety. A well-designed system should not only function efficiently but also align with these broader considerations to ensure responsible usage.

8.1 Social Aspects

The proposed system plays an important role in improving public transportation services, especially in small cities where digital infrastructure is still developing. By providing real-time information about bus locations, the system directly enhances the daily experience of commuters.

One of the key benefits is the reduction of uncertainty in travel. Passengers no longer need to rely on fixed schedules or guesswork, as they can view the live position of buses. This leads to better time management and reduces waiting time at bus stops. Over time, such improvements can increase public trust in transportation systems and encourage more people to use public transport.

The system also contributes to operational efficiency from an administrative perspective. Transport authorities can monitor vehicle movement and identify delays or irregularities in service. This enables better decision-making and improves overall service quality.

In terms of accessibility, the system is designed to be web-based, making it usable on commonly available devices such as smartphones and computers. This eliminates the need for installing additional applications and allows a broader group of users to benefit from the system.

However, certain limitations must be considered. Not all individuals may have access to internet-enabled devices, which creates a digital gap. Additionally, users may become overly dependent on the system, and any failure or incorrect data could lead to inconvenience. There is also a gradual reduction in human interaction, as digital systems replace traditional inquiry methods.

In summary, the social impact of the system can be viewed as a balance between improved convenience and the need to ensure inclusivity and reliability. The system supports the movement towards smarter urban environments while promoting efficient use of public transport.

8.2 Legal Aspects

From a legal standpoint, the system must comply with regulations related to data handling, system security, and user rights. Although the current implementation uses simulated data, future integration with GPS-based systems will involve the collection and processing of real-time location data.

A major legal concern in such systems is data privacy. Location data, even when related to public transport, must be handled carefully to avoid misuse. Regulations such as the Digital Personal Data Protection Act in India emphasize the importance of lawful data collection, user consent, and secure storage of information. Any real-world implementation must ensure that these principles are followed.

Another important aspect is accountability. The responsibility for maintaining accurate and reliable information lies with system administrators and transport authorities. Incorrect data, such as wrong bus locations or delayed updates, can mislead users and affect their travel decisions. Therefore, mechanisms must be in place to ensure data accuracy and timely updates.

Legal risks may also arise from unauthorized access or system misuse. For instance, if an attacker gains control of the system and manipulates bus data, it could disrupt services and create confusion. To prevent such situations, the system must include proper security measures such as authentication, controlled access, and secure communication protocols.

Key legal considerations include:

- Protection of user and system data
- Compliance with national data protection laws
- Ensuring accountability for system outputs
- Prevention of unauthorized access and misuse

As the system evolves, additional legal challenges may arise, particularly in defining responsibility for service failures and managing real-time data ownership.

8.3 Ethical Aspects

Ethical considerations are central to the responsible development of this system. As an engineering solution designed for public use, the primary objective is to ensure that the system benefits users without causing harm or inconvenience.

One of the most important ethical aspects is the accuracy of information. Since users rely on the system for travel decisions, any incorrect or outdated data can negatively affect them. Developers must therefore ensure that the system processes and displays information as accurately as possible.

Another ethical factor is accessibility and fairness. The system should be designed in a way that it is easy to use for all users, regardless of their technical background. A simple and intuitive interface ensures that even first-time users can navigate the system without difficulty.

Privacy also plays a significant role in ethical considerations. While the current system uses simulated data, future implementations involving real GPS tracking must ensure that sensitive information is not exposed or misused. Ethical system design requires that user data is protected and used only for intended purposes.

From a professional perspective, engineers must follow ethical standards during development. This includes:

- Being transparent about system capabilities and limitations
- Avoiding manipulation or misrepresentation of results

- Ensuring thorough testing before deployment
- Maintaining honesty in reporting system performance

The system does not introduce addictive behaviour or negatively impact human relationships. Instead, it complements existing transport systems by improving efficiency and convenience.

8.4 Sustainability Aspects

Sustainability in this project is primarily associated with efficient resource utilization and its indirect contribution to environmental improvement.

Since the system is largely software-based, it does not require extensive physical hardware during development. This reduces the generation of electronic waste and minimizes the environmental impact in the initial stages. The use of open-source tools further supports sustainable development by reducing dependency on proprietary resources.

A major contribution of the system lies in promoting public transportation. When users have access to real-time tracking, they are more likely to choose buses over private vehicles. This shift can lead to reduced fuel consumption, lower carbon emissions, and decreased traffic congestion.

From a design perspective, the system is lightweight and efficient. It operates with minimal computational resources and can run on standard devices, which helps in reducing energy consumption. Efficient handling of data and optimized communication further contribute to energy savings.

Sustainability aspects of the system can be summarized as follows:

- Reduced environmental impact due to minimal hardware usage
- Encouragement of eco-friendly transportation choices
- Efficient use of computational and energy resources
- Support for long-term scalable system design

In future developments, sustainability can be enhanced by integrating route optimization algorithms, improving fleet efficiency, and adopting energy-efficient cloud infrastructure.

8.5 Safety Aspects

Safety is a critical factor in ensuring that the system operates reliably and does not cause harm to users or infrastructure. In this project, safety is primarily related to data security, system stability, and accurate information delivery.

One of the main concerns is secure communication between the server and clients. Since the system relies on real-time data transmission, it is important to ensure that data is protected from unauthorized access or manipulation. Implementing secure communication protocols and authentication mechanisms can significantly improve system safety.

Another important aspect is system reliability. The system should function consistently without crashes or unexpected behaviour, even when multiple users are connected simultaneously. Reliable performance ensures that users can depend on the system for accurate information.

The system must also handle unexpected events effectively. For example, when a bus reports a breakdown, the system should update its status immediately and reflect this change across all user interfaces. This prevents confusion and helps users make informed decisions.

Future safety improvements may include:

- Validation of GPS data to avoid incorrect positioning
- Handling network failures without loss of critical data
- Implementation of backup systems for continuous operation
- Monitoring and detection of abnormal system behaviour

From the user's perspective, safety is closely linked to trust. Providing clear, accurate, and timely information ensures that users can rely on the system without hesitation.

Overall, the project successfully integrates technical functionality with broader considerations such as social impact, legal compliance, ethical responsibility, sustainability, and safety. While the current implementation is based on simulated data, it establishes a strong foundation for real-world deployment. With further enhancements and adherence to standards, the system can evolve into a reliable and scalable solution for improving public transportation systems.

Chapter 9

Conclusion

The Real-Time Public Transport Tracking System developed in this project demonstrates an effective approach to improving public transportation monitoring using modern web technologies. The primary objective of the project was to design a system capable of providing real-time location updates of buses, along with additional information such as speed, capacity, and operational status. This objective has been successfully achieved through the integration of a backend server, real-time communication using Socket.IO, and a map-based visualization interface.

The system was designed to simulate real-world conditions by generating dynamic bus data instead of relying on physical GPS hardware. This approach allowed the development and testing of core functionalities such as continuous location updates, real-time data transmission, and multi-user synchronization. The implementation ensures that all connected users receive consistent and updated information, thereby fulfilling the requirement of real-time tracking.

The results obtained from testing indicate that the system performs reliably under simulated conditions. The periodic updates of bus data are reflected accurately on the map interface, and the delay in communication remains minimal. Features such as adding or removing buses, updating trip status, and handling breakdown conditions were implemented successfully and verified through different test scenarios. These results confirm that the system meets its intended objectives of providing a responsive and user-friendly transport tracking solution.

In addition to meeting functional requirements, the project also demonstrates scalability and flexibility in design. The use of modular architecture allows easy integration of new features without major changes to the existing system. The separation of backend logic and frontend visualization ensures better maintainability and future expansion.

Future Scope

While the current implementation achieves its core objectives, there are several areas where the system can be further improved and extended to make it more practical and robust for real-world deployment.

One of the most important enhancements is the integration of actual GPS hardware. Replacing simulated data with real-time GPS inputs would significantly improve the accuracy and usability of the system. This would allow the system to be deployed in real transportation environments.

Another area of improvement is the implementation of route-based tracking. In the current system, bus movement is simulated randomly. Introducing predefined routes and stops would make the system more realistic and useful for passengers.

The system can also be enhanced by incorporating a database to store historical data. This would enable features such as:

- Travel history tracking
- Performance analysis of routes
- Predictive arrival time estimation

Scalability is another aspect that can be improved. As the number of users increases, techniques such as load balancing and cloud deployment can be used to maintain system performance and reduce latency.

From a user perspective, the interface can be further refined by adding features such as:

- Estimated time of arrival (ETA)
- Notifications for delays or breakdowns
- Mobile-friendly or dedicated application interface

Security enhancements can also be considered, including authentication mechanisms and encrypted communication to protect data from unauthorized access.

Overall, the project successfully demonstrates how a real-time tracking system can be developed using software-based solutions without relying on hardware in the initial stage. It provides a strong foundation for future improvements and real-world implementation. With further enhancements, the system has the potential to become a practical solution for improving public transportation efficiency, especially in small and developing cities

References

- [1] J. Babu et al., “GPS Based Bus Tracking System,” International Journal of Engineering Research & Technology (IJERT), vol. 9, no. 13, 2021.
- [2] Y. H. A. Abdelhafed, “Improving Transportation Management by Real Time Bus Tracking using GPS,” IJRASET, 2025.
- [3] G. Gayathri et al., “Real Time Bus Tracking System,” International Journal of Advanced Research in Computer Science, vol. 9, 2018.
- [4] B. Thomas et al., “An Intelligent Location Tracker for Public Transport,” IJERT, vol. 10, no. 4, 2021.
- [5] K. Gogri et al., “Real Time Bus Tracking System,” IJERT, vol. 9, no. 6, 2020.
- [6] H. Khaled et al., “Intelligent Transportation System Real-Time Tracking,” Qeios, 2023.
- [7] J. C. Ogbonna et al., “Real-Time Public Vehicle Mobile Tracking System Using GPS Technology,” International Journal of Computer Trends and Technology, vol. 38, no. 1, pp. 71–80, 2016.
- [8] I. Madhur et al., “Smart Bus Tracking and Management System,” Iconic Research and Engineering Journals, vol. 1, pp. 80–84, 2018.
- [9] A. Bessa et al., “RioBusData: Outlier Detection in Bus Routes,” arXiv preprint arXiv:1601.06128, 2016.
- [10] K. Nguyen et al., “Los Angeles Metro Bus Data Analysis Using GPS Trajectory,” arXiv preprint arXiv:1909.00955, 2019.

OpenStreetMap contributors, 2024. *OpenStreetMap: Collaborative mapping platform*.

Available at: <https://www.openstreetmap.org>

Agafonkin, V., 2023. *Leaflet: An open-source JavaScript library for mobile-friendly interactive maps*. Available at: <https://leafletjs.com>

Socket.IO, 2024. *Realtime application framework for Node.js*. Available at: <https://socket.io>

Appendix

Appendix A: Project GitHub

https://github.com/jovial123keerthi-bit/Real-Time-Public-Transport-Tracking-for-Small-Cities-CSG_45-

Appendix B: Project Report-Similarity Report

A similarity analysis of the project report was conducted using Turnitin to ensure the originality of the work. The plagiarism summary and overall similarity percentage are shown in Fig B.1, confirming a low similarity index.

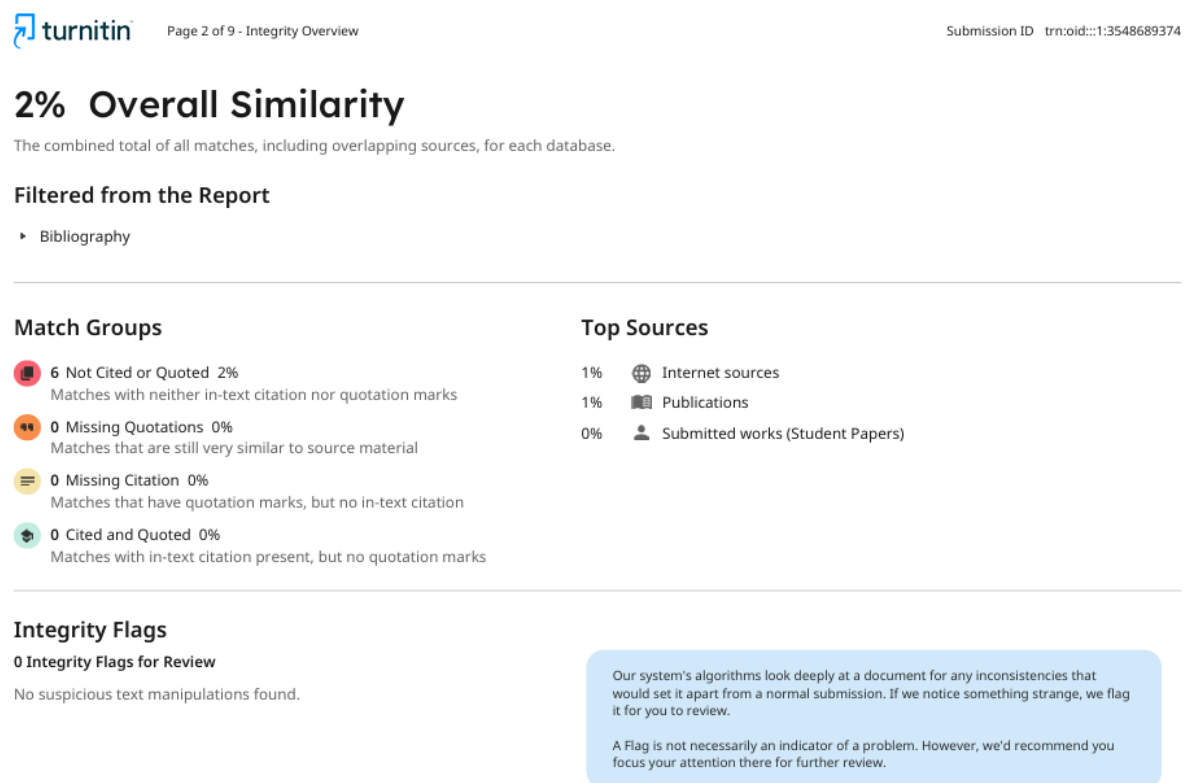


Fig B.1 Project Report Similarity Analysis–Turnitin Overview

Turnitin analysis further provides detailed matching sources and highlighted similarities, as seen in Fig B.2. It lists the percentage of matches with external sources, academic papers, and internet content

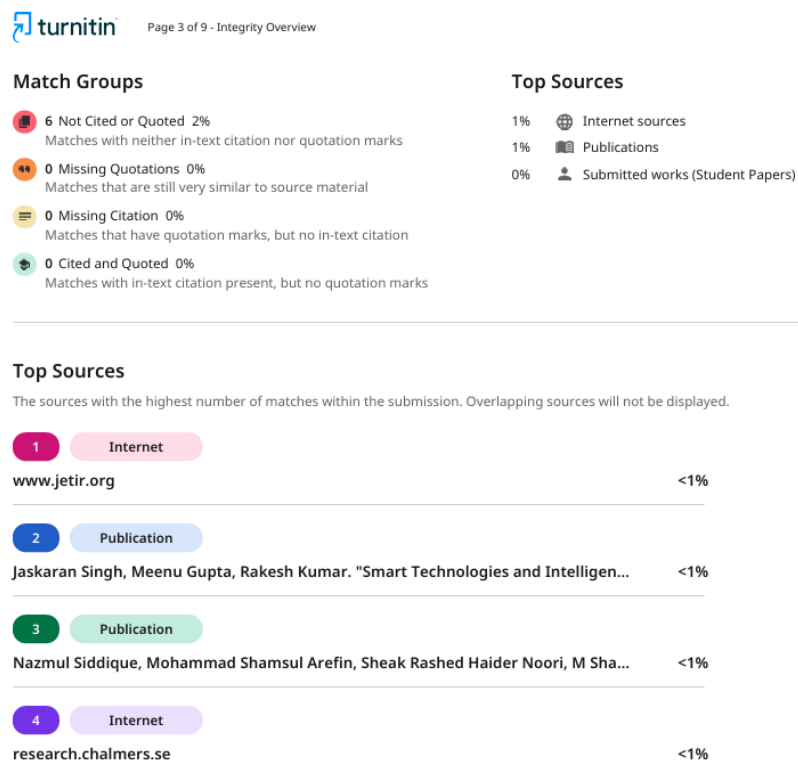


Fig B.2 Detailed Turnitin Source and Similarity Breakdown Report

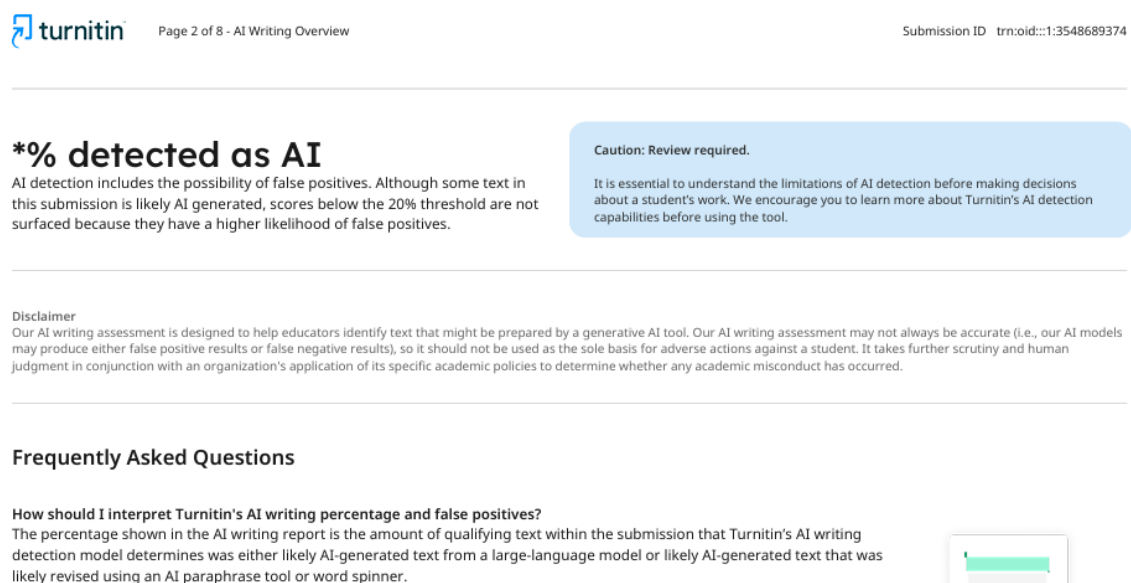


Fig B.3 Detailed Turnitin AI detection Score

Appendix C : Images of project

This dashboard displays live information for multiple buses, including route number, speed, passenger capacity, estimated arrival time (ETA), and crowd status (low, medium, or full). overview for administrators to monitor fleet conditions in real time is displayed in Fig C.1.

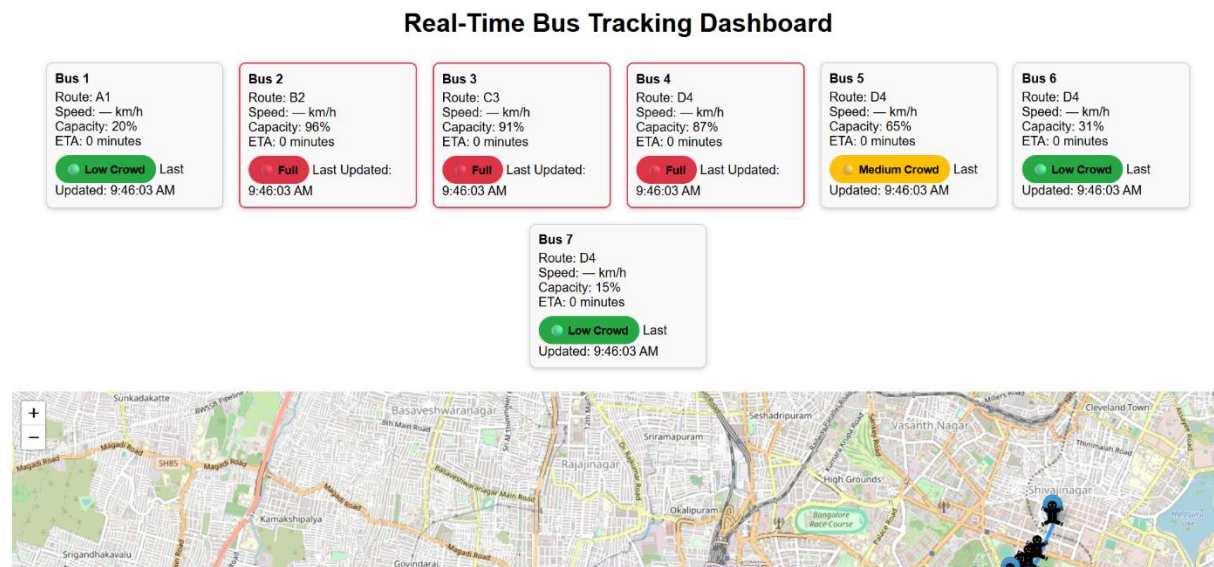


Fig C.1 Real-Time Bus Tracking Dashboard

This map interface in Fig C.2 shows the real-time locations of buses along their routes, marked with icons and connected by route paths. It helps visualize movement and track bus progression across the city.



Fig C.2 Live Bus Route Visualization on Map

This view in Fig C.3 highlights a selected bus stop on the map, displaying additional information such as stop name and nearby route details, enabling users to understand bus positions relative to stops.

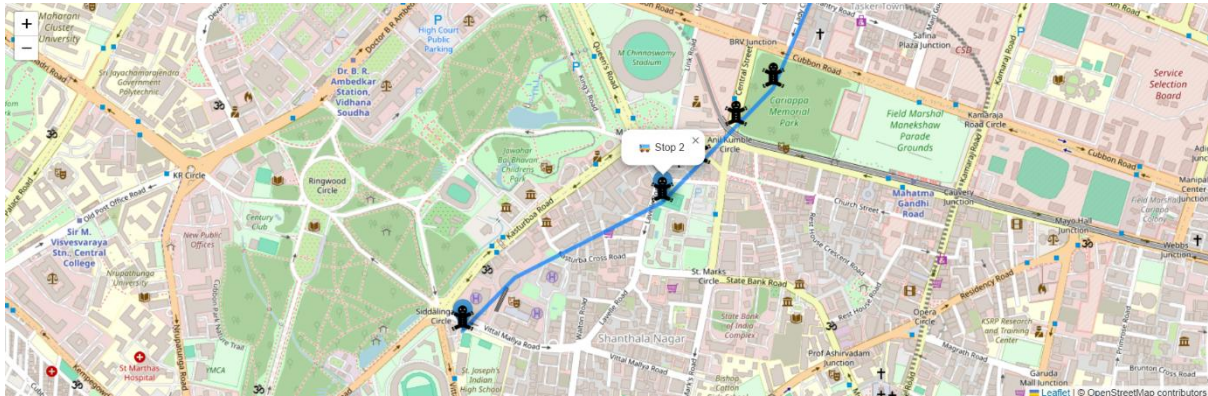


Fig C.3 Bus Stop Interaction and Route Details

This login screen in Fig C.4 allows administrators to securely access the system using credentials. Admin users can manage routes, monitor buses, and oversee system operations.

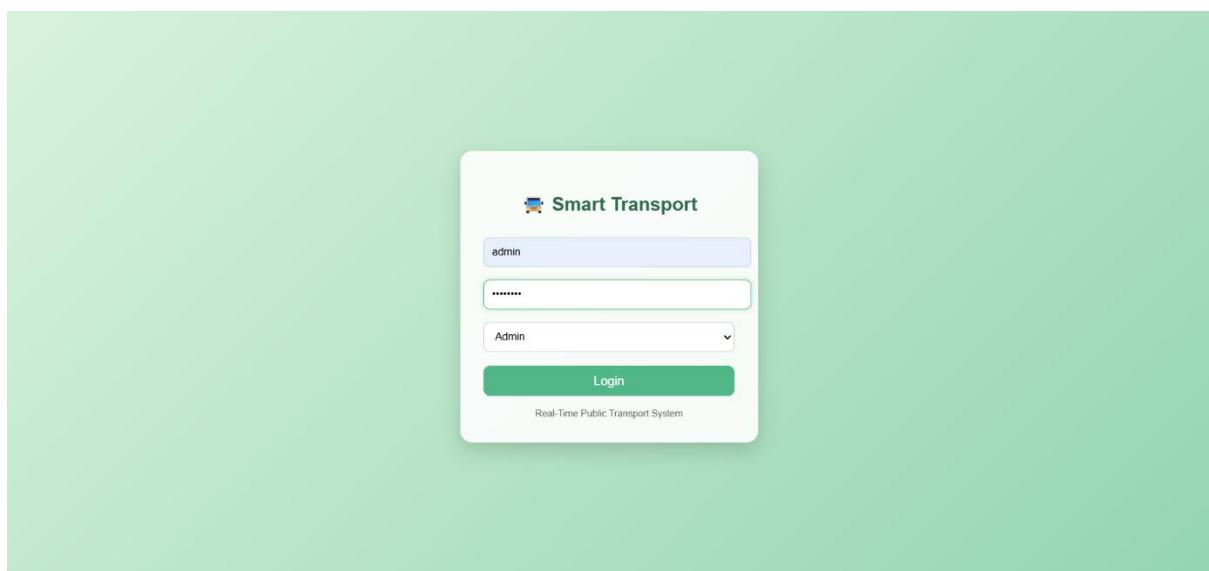
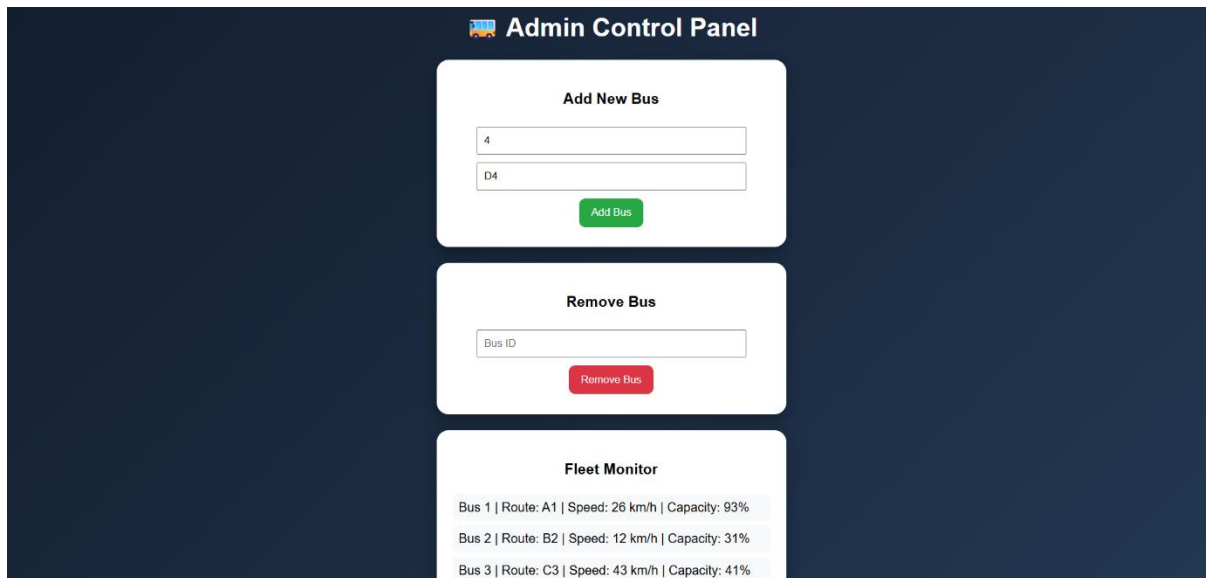


Fig C.4 Admin Login Interface

This interface in Fig C.5 represents the admin control panel of the system, where administrators can manage the fleet. It allows adding new buses by entering bus details, removing existing buses using their ID, and monitoring all active buses in real time. The fleet monitor section displays key parameters such as route, speed, and passenger capacity, enabling efficient oversight of public transport operations.

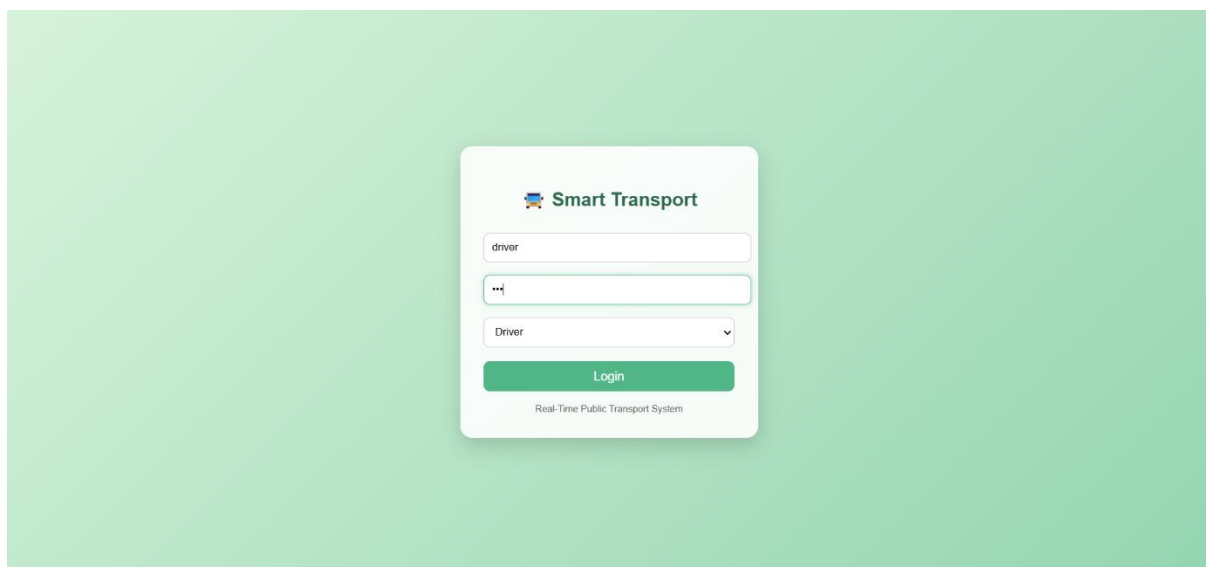


The Admin Control Panel interface features a dark blue background with three white panels. The top panel, titled 'Add New Bus', contains two input fields (one with '4' and another with 'D4') and a green 'Add Bus' button. The middle panel, titled 'Remove Bus', has a 'Bus ID' input field and a red 'Remove Bus' button. The bottom panel, titled 'Fleet Monitor', displays a table of bus data:

Bus ID	Route	Speed (km/h)	Capacity (%)
Bus 1	A1	26	93%
Bus 2	B2	12	31%
Bus 3	C3	43	41%

Fig C. 5 Admin Control Panel Interface

This interface in Fig C.6 is designed for drivers to log into the system. It enables them to update bus status, location, and other operational data in real time.



The Driver Login Interface is centered on a light green background. It features a white login card with the title 'Smart Transport' and a bus icon. The card includes three input fields: a text field with 'driver', a password field with '***', and a dropdown menu labeled 'Driver'. Below these is a green 'Login' button. At the bottom of the card, it reads 'Real-Time Public Transport System'.

Fig C.6 Driver Login Interface

This interface in Fig C.7 shows the driver dashboard, designed for bus operators to manage trips. Drivers can select their assigned bus, start and end trips, and report breakdowns if necessary. The system also provides real-time status updates, indicating whether the bus is currently in operation, ensuring smooth communication between drivers and the control system .

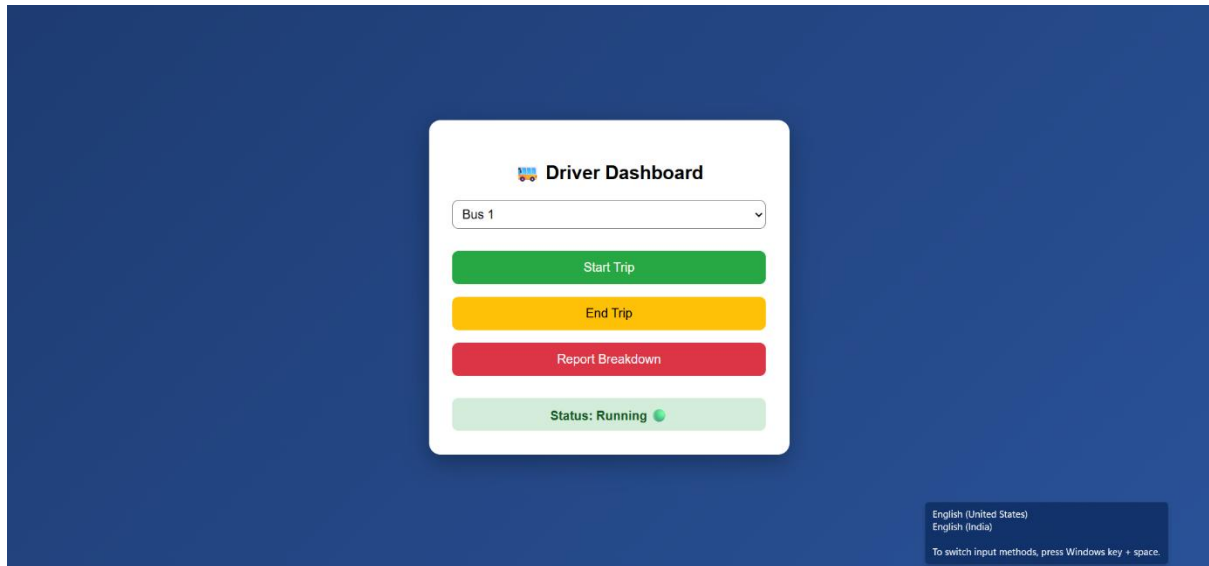


Fig C.7 Driver Dashboard Interface